Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour

Priya Goyal Piotr Dollár Ross Girshick Pieter Noordhuis
Lukasz Wesolowski Aapo Kyrola Andrew Tulloch Yangqing Jia Kaiming He

Facebook

Abstract

Deep learning thrives with large neural networks and large datasets. However, larger networks and larger datasets result in longer training times that impede research and development progress. Distributed synchronous SGD offers a potential solution to this problem by dividing SGD minibatches over a pool of parallel workers. Yet to make this scheme efficient, the per-worker workload must be large, which implies nontrivial growth in the SGD minibatch size. In this paper, we empirically show that on the ImageNet dataset large minibatches cause optimization difficulties, but when these are addressed the trained networks exhibit good generalization. Specifically, we show no loss of accuracy when training with large minibatch sizes up to 8192 images. To achieve this result, we adopt a hyper-parameter-free linear scaling rule for adjusting learning rates as a function of minibatch size and develop a new warmup scheme that overcomes optimization challenges early in training. With these simple techniques, our Caffe2-based system trains ResNet-50 with a minibatch size of 8192 on 256 GPUs in one hour, while matching small minibatch accuracy. Using commodity hardware, our implementation achieves ~90% scaling efficiency when moving from 8 to 256 GPUs. Our findings enable training visual recognition models on internet-scale data with high efficiency.

1. Introduction

Scale matters. We are in an unprecedented era in AI research history in which the increasing data and model scale is rapidly improving accuracy in computer vision [22, 41, 34, 35, 36, 16], speech [17, 40], and natural language processing [7, 38]. Take the profound impact in computer vision as an example: visual representations learned by deep convolutional neural networks [23, 22] show excellent performance on previously challenging tasks like ImageNet classification [33] and can be transferred to difficult perception problems such as object detection and segmentation [8, 10, 28]. Moreover, this pattern generalizes: larger datasets and neural network architectures consistently yield improved accuracy across all tasks that benefit from pre-training [22, 41, 34, 35, 36, 16]. But as model and data scale grow, so does training time; discovering the potential and limits of large-scale deep learning requires developing novel techniques to keep training time manageable.

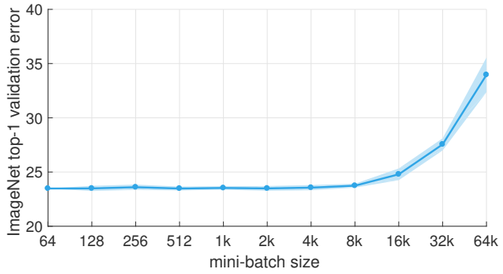


Figure 1. **ImageNet top-1 validation error *vs*. minibatch size.** Error range of plus/minus *two* standard deviations is shown. We present a simple and general technique for scaling distributed synchronous SGD to minibatches of up to 8k images *while maintaining the top-1 error of small minibatch training*. For all minibatch sizes we set the learning rate as a *linear* function of the minibatch size and apply a simple warmup phase for the first few epochs of training. All other hyper-parameters are kept fixed. Using this simple approach, accuracy of our models is invariant to minibatch size (up to an 8k minibatch size). Our techniques enable a linear reduction in training time with ~90% efficiency as we scale to large minibatch sizes, allowing us to train an accurate 8k minibatch ResNet-50 model in 1 hour on 256 GPUs.

The goal of this report is to demonstrate the feasibility of, and to communicate a practical guide to, large-scale training with distributed *synchronous* stochastic gradient descent (SGD). As an example, we scale ResNet-50 [16] training, originally performed with a minibatch size of 256 images (using 8 Tesla P100 GPUs, training time is 29 hours), to larger minibatches (see Figure 1). In particular, we show that *with a large minibatch size of 8192, we can train ResNet-50 in 1 hour using 256 GPUs while maintaining*

高精度大批量随机梯度下降算法：1小时内训练ImageNet数据集

Priya Goyal、Piotr Doll´ar、Ross Girshick、Pieter Noordhuis、Lukasz Wesolowski、Aapo Kyrola、Andrew Tulloch、Yangqing Jia、Kaiming He

Facebook

摘要

深度学习依赖于大规模神经网络和海量数据集才能蓬勃发展。然而，更大的网络与数据集会带来更长的训练时间，从而阻碍研究与开发的进展。分布式同步随机梯度下降通过将随机梯度下降的小批量任务分配到多个并行工作节点上，为解决这一问题提供了可能。但要让该方案高效运行，每个工作节点所需承担的任务量必须较大，这又意味着随机梯度下降小批量大小需要相应地大幅增加。在本文中，我们通过实证发现，在*ImageNet*数据集上使用大批量训练虽然会带来优化难题，但只要解决了这些问题，训练出的网络仍能具备良好的泛化能力。具体而言，当使用高达*8192*张图像的大批量进行训练时，模型的平均精度并不会出现下降。为达成这一成果，我们采用了一种无需超参数的线性缩放规则，根据小批量大小动态调整学习率，同时还设计了一种新的预热方案，能够在训练初期就克服优化方面的挑战。借助这些简单有效的技巧，基于*Caffe2*框架的系统仅需*256*块图形处理器，配合*8192*大小的批量训练，就能在*1*小时内训练出*ResNet-50*模型，且其性能可与小批量训练的结果相媲美。即便使用普通硬件，我们的实现方案在将*GPU*数量从*8*块增加到*256*块时，也能实现 *~90%*的效率提升。这些研究结果为高效训练基于互联网规模数据的视觉识别模型奠定了基础。

1. 引言

规模至关重要。在人工智能研究史上，我们正处于一个前所未有的时代——日益增长的数据量和模型规模正在快速提升计算机视觉领域的平均精度[22, 41, 34, 35, 36, 16], 语音处理 [17, 40], 以及自然语言处理 [7, 38]。以计算机视觉领域的显著进步为例：通过深度卷积神经网络学习得到的视觉表征 [23, 22]在诸如ImageNet分类任务这类此前颇具挑战性的任务中表现出色 [33]，同时还能被应用到物体检测和图像分割等复杂的感知问题中

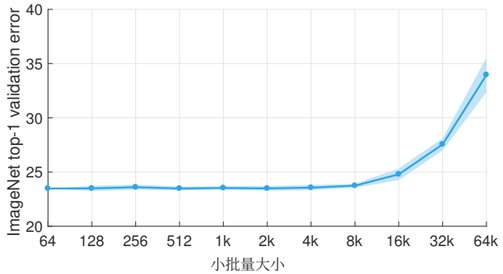


图1. **ImageNet的Top-1验证误差与小批量大小**的关系。图中显示了误差在±两个标准差范围内的变化情况。我们提出了一种简单且通用的方法，用于将分布式同步随机梯度下降算法扩展到最多包含8k张图像的小批量数据上，同时保持小批量训练时的*Top-1*误差水平。对于所有不同的小批量大小，我们都将学习率设定为该小批量大小的线性函数，并在训练的前几轮加入简单的预热阶段。其余所有超参数则保持不变。通过这种简单的方法，我们的模型在各种小批量大小下（最高可达8k批量大小）的准确率都不会发生变化。借助这些技术，随着我们将训练规模扩大到更大的小批量数据，训练时间能够实现线性减少，且效率还能提升 ~90%，这使我们能够在256台图形处理器上仅用1小时就训练出精度较高的8k批量大小ResNet-50模型。

此外，这种趋势具有普遍性：规模更大的数据集和神经网络架构总能为所有可从预训练中受益的分类任务带来更高的平均精度 [22, 41, 34, 35, 36, 16]。不过，随着模型和数据规模的扩大，训练时间也会随之增加；要探索大规模深度学习的潜力与局限，就必须开发新的技术手段来控制训练时间，使其处于可管理的范围之内。

本报告旨在展示利用分布式同步随机梯度下降（SGD）进行大规模训练的可行性，并提供相应的实用指导。作为示例，我们将原本采用256张图像作为小批量大小、并借助8台Tesla P100显卡需要29小时才能完成训练的ResNet-50 [16]训练任务，调整为使用更大的小批量数据（详见图1）。具体而言，我们证明了：当使用8192这样较大的小批量大小时，即便动用256台GPU，也依然能够在1小时内完成ResNet-50模型的训练，同时还能保持与原有方法相同的准确率。

the same level of accuracy as the 256 minibatch baseline. While distributed synchronous SGD is now commonplace, no existing results show that generalization accuracy can be maintained with minibatches as large as 8192 or that such high-accuracy models can be trained in such short time.

To tackle this unusually large minibatch size, we employ a simple and hyper-parameter-free *linear scaling rule* to adjust the learning rate. While this guideline is found in earlier work [21, 4], its empirical limits are not well understood and informally we have found that it is not widely known to the research community. To successfully apply this rule, we present a new *warmup* strategy, *i.e.*, a strategy of using lower learning rates at the start of training [16], to overcome early optimization difficulties. Importantly, not only does our approach match the baseline *validation* error, *but also yields training error curves that closely match the small minibatch baseline.* Details are presented in §2.

Our comprehensive experiments in §5 show that *optimization* difficulty is the main issue with large minibatches, rather than poor *generalization* (at least on ImageNet), in contrast to some recent studies [20]. Additionally, we show that the linear scaling rule and warmup generalize to more complex tasks including object detection and instance segmentation [9, 31, 14, 28], which we demonstrate via the recently developed Mask R-CNN [14]. We note that a robust and successful guideline for addressing a wide range of minibatch sizes has not been presented in previous work.

While the strategy we deliver is simple, its successful application requires correct implementation with respect to seemingly minor and often not well understood implementation details within deep learning libraries. Subtleties in the implementation of SGD can lead to incorrect solutions that are difficult to discover. To provide more helpful guidance we describe common pitfalls and the relevant implementation details that can trigger these traps in §3.

Our strategy applies regardless of framework, but achieving efficient linear scaling requires nontrivial communication algorithms. We use the open-source *Caffe2*[1] deep learning framework and *Big Basin* GPU servers [24], which operates efficiently using standard Ethernet networking (as opposed to specialized network interfaces). We describe the systems algorithms that enable our approach to operate near its full potential in §4.

The practical advances described in this report are helpful across a range of domains. In an industrial domain, our system unleashes the potential of training visual models from internet-scale data, enabling training with *billions of images per day*. Of equal importance, in a research domain, we have found it to simplify migrating algorithms from a single-GPU to a multi-GPU implementation without requiring hyper-parameter search, *e.g.* in our experience migrating Faster R-CNN [31] and ResNets [16] from 1 to 8 GPUs.

[1] http://www.caffe2.ai

2. Large Minibatch SGD

We start by reviewing the formulation of Stochastic Gradient Descent (SGD), which will be the foundation of our discussions in the following sections. We consider supervised learning by minimizing a loss $L(w)$ of the form:

$$L(w) = \frac{1}{|X|} \sum_{x \in X} l(x, w). \tag{1}$$

Here w are the weights of a network, X is a labeled training set, and $l(x, w)$ is the loss computed from samples $x \in X$ and their labels y. Typically l is the sum of a classification loss (*e.g.*, cross-entropy) and a regularization loss on w.

Minibatch Stochastic Gradient Descent [32], usually referred to as simply as SGD in recent literature even though it operates on minibatches, performs the following update:

$$w_{t+1} = w_t - \eta \frac{1}{n} \sum_{x \in \mathcal{B}} \nabla l(x, w_t). \tag{2}$$

Here $\mathcal{B}$ is a *minibatch* sampled from X and $n = |\mathcal{B}|$ is the *minibatch size*, η is the *learning rate*, and t is the iteration index. Note that in practice we use momentum SGD; we return to a discussion of momentum in §3.

2.1. Learning Rates for Large Minibatches

Our goal is to use large minibatches in place of small minibatches while *maintaining training and generalization accuracy*. This is of particular interest in distributed learning, because it can allow us to scale to multiple workers[2] using simple data parallelism without reducing the per-worker workload and without sacrificing model accuracy.

As we will show in comprehensive experiments, we found that the following learning rate scaling rule is surprisingly effective for a broad range of minibatch sizes:

> ***Linear Scaling Rule:*** *When the minibatch size is multiplied by k, multiply the learning rate by k.*

All other hyper-parameters (weight decay, *etc*.) are kept unchanged. As we will show in §5, the *linear scaling rule* can help us to not only match the accuracy between using small and large minibatches, but equally importantly, to largely match their training curves, which enables rapid debugging and comparison of experiments prior to convergence.

Interpretation. We present an informal discussion of the linear scaling rule and why it may be effective. Consider a network at iteration t with weights w_t, and a sequence of k minibatches $\mathcal{B}_j$ for $0 \le j < k$ each of size n. We compare the effect of executing k SGD iterations with *small minibatches* $\mathcal{B}_j$ and learning rate η versus a single iteration with a *large minibatch* $\cup_j \mathcal{B}_j$ of size kn and learning rate $\hat{\eta}$.

[2] We use the terms 'worker' and 'GPU' interchangeably in this work, although other implementations of a 'worker' are possible. 'Server' denotes a set of 8 GPUs that does not require communication over a network.

其准确率可与256样本小批量基线方法相媲美。虽然分布式同步随机梯度下降目前已十分常见，但现有的研究结果并未表明，使用高达8192的小批量大小依然能够保持模型的泛化能力，更未曾有研究证明可以在如此短的时间内训练出具有如此高准确率的模型。

为了解决这种异常大的小批量大小问题，我们采用了一种简单且无需超参数调整的线性缩放规则来对学习率进行相应调整。尽管这一方法早前就有相关研究提及，但其实际适用范围至今仍不十分明确，而且据我们非正式观察，科研界对此也知之甚少。为确保该规则能够得到有效应用，我们提出了一种新的预热阶段策略，也就是说，即在训练初期使用较低的学习率，以此克服早期的优化难题。值得注意的是，我们的方法不仅能够与基准曲线的验证误差保持一致，其生成的训练误差曲线也与小样本小批量基线极为接近。详细内容见§2。

我们在§5中进行的全面实验表明优化方面的困难才是大批量训练面临的主要问题，而非泛化能力不足（至少在ImageNet数据集上如此），这与部分近期研究得出的结论有所不同。此外，我们还证明了这种线性缩放规则以及预热阶段策略同样适用于包括物体检测和实例分割在内的更复杂任务，并且我们通过最新开发的Mask R-CNN模型对这一点进行了验证。需要指出的是，以往的研究中尚未出现能够有效应对各种不同批量大小的稳健指导原则。

虽然我们所采用的策略十分简单，但要使其成功应用，就必须准确实现深度学习框架中那些看似细微且往往不易理解的实施细节。随机梯度下降算法在实现过程中的种种微妙差异，都可能导致出现难以发现的错误结果。为提供更有价值的指导，我们将在§部分详细阐述常见的陷阱以及可能引发这些问题的相关实现细节。3。

无论使用何种深度学习框架，我们的策略都能适用，但要想实现高效的线性扩展，就需要采用相当复杂的通信算法。我们采用了开源的*Caffe2*[1]深度学习框架*BigBasin* 图形处理器服务器 [24]，这类服务器借助标准以太网进行高效通信（而非专用网络接口）。我们将在§部分介绍那些能够让我们的方法发挥出接近最大性能的系统算法。4。

本报告中所介绍的各类实用进展在众多领域都具有显著价值。在工业领域，我们的系统能够充分挖掘基于互联网规模数据训练视觉模型的潜力，从而实现每天处理数十亿张图像的训练效率。同样重要的是，在研究领域，这种方法简化了将算法从单GPU架构迁移至多GPU架构的过程，且无需进行超参数搜索，例如，根据我们的经验，将Faster R-CNN [31] 以及ResNet系列模型 [16] 从1块GPU升级到8块GPU即可完成迁移。

[1] http://www.caffe2.ai

2. 大批量SGD算法

首先，我们将回顾随机梯度下降（SGD）的原理，这也是后续章节讨论的基础。在这里，我们通过最小化如下形式的损失 $L(w)$来探讨有监督学习问题：

$$L(w) = \frac{1}{|X|} \sum_{x \in X} l(x, w). \tag{1}$$

此处 w 为网络中的权重， X 是带标签的训练集，而 $l(x,\ w)$则是根据样本 $x \in X$及其对应标签 y计算得出的损失值。通常情况下， l 是由分类损失值（如例如交叉熵）以及对 w施加的正则化损失值相加而成的。

小批量随机梯度下降 [32], 在近期的相关研究中，尽管该算法是在小批量数据上运行的，但通常仍被简称为随机梯度下降（SGD），其更新规则如下：

$$w_{t+1} = w_t - \eta \frac{1}{n} \sum_{x \in \mathcal{B}} \nabla l(x, w_t). \tag{2}$$

此处 $\mathcal{B}$ 是一个从小批量数据中抽取的样本，它是小批量大小； η 是学习率，而 t 则是迭代次数。需要注意的是，在实际应用中我们通常采用动量随机梯度下降；关于动量的详细讨论可见第3节。

2.1. 大批量训练时的学习率设置

我们的目标是在保持训练效果与泛化精度的同时，用大批量训练替代小批量训练。这在分布式学习领域尤为重要，因为它使我们能够通过简单的数据并行方式扩展到多个工作节点[2]，既不会增加每个工作节点的处理负担，也不会降低模型的精度。

正如我们在一系列详尽实验中所展示的那样，对于多种不同的小批量大小而言，以下学习率缩放规则具有出人意料的良好效果：

> 线性缩放规则：当小批量大小乘以 k时，学习率也应相应地乘以 k。

其余所有的超参数（如权重衰减等）均保持不变。正如我们将在§5中所示，线性缩放规则不仅能帮助我们在使用小批量数据与大批量训练时获得相近的平均精度，更为重要的是，它还能让两者的训练曲线高度相似，从而便于在收敛之前快速调试并对比实验结果。

解释。 本文将对线性缩放规则及其为何能有效发挥作用进行简要阐述。假设在迭代 t 时存在一个权重为 w_t的网络，同时有 k 组小批量数据 $\mathcal{B}_j$ ，每组的大小均为 n。我们会比较执行 k 次随机梯度下降迭代——其中采用较小的小批量 $\mathcal{B}_j$ 且学习率为 η ——与仅执行一次迭代但使用较大的小批量 $\cup_j \mathcal{B}_j$ （大小为 kn ，学习率为$\hat{\eta}$）时的效果差异。

[2] 在本研究中，我们将‘工作节点’与‘图形处理器’这两个术语视为可互换使用的，不过实际上也存在其他类型的工作节点实现方式。而‘服务器’则指由8块图形处理器组成的、无需通过网络进行通信的硬件集群。

According to (2), after k iterations of SGD with learning rate η and a minibatch size of n we have:

$$w_{t+k} = w_t - \eta\frac{1}{n}\sum_{j<k}\sum_{x\in\mathcal{B}_j}\nabla l(x, w_{t+j}). \quad (3)$$

On the other hand, taking a single step with the large minibatch $\cup_j\mathcal{B}_j$ of size kn and learning rate $\hat{\eta}$ yields:

$$\hat{w}_{t+1} = w_t - \hat{\eta}\frac{1}{kn}\sum_{j<k}\sum_{x\in\mathcal{B}_j}\nabla l(x, w_t). \quad (4)$$

As expected, the updates differ, and it is *unlikely* that $\hat{w}_{t+1} = w_{t+k}$. However, if we *could* assume $\nabla l(x, w_t) \approx \nabla l(x, w_{t+j})$ for $j < k$, then setting $\hat{\eta} = k\eta$ would yield $\hat{w}_{t+1} \approx w_{t+k}$, and the updates from small and large minibatch SGD would be similar. Although this is a strong assumption, we emphasize that if it were true the two updates are similar *only* if we set $\hat{\eta} = k\eta$.

The above interpretation gives intuition for one case where we may hope the linear scaling rule to apply. In our experiments with $\hat{\eta} = k\eta$ (and warmup), small and large minibatch SGD not only result in models with the same final accuracy, but also, the training curves match closely. Our empirical results suggest that the above approximation *might* be valid in large-scale, real-world data.

However, there are at least two cases when the condition $\nabla l(x, w_t) \approx \nabla l(x, w_{t+j})$ will clearly not hold. First, in initial training when the network is changing rapidly, it does not hold. We address this by using a warmup phase, discussed in §2.2. Second, minibatch size cannot be scaled indefinitely: while results are stable for a large range of sizes, beyond a certain point accuracy degrades rapidly. Interestingly, this point is as large as ~8k in ImageNet experiments.

Discussion. The above linear scaling rule was adopted by Krizhevsky [21], if not earlier. However, Krizhevsky reported a 1% increase of error when increasing the minibatch size from 128 to 1024, whereas we show how to maintain accuracy across a much broader regime of minibatch sizes. Chen *et al*. [5] presented a comparison of numerous distributed SGD variants, and although their work also employed the linear scaling rule, it did not establish a small minibatch baseline. Li [25] (§4.6) showed distributed ImageNet training with minibatches up to 5120 without a loss in accuracy after convergence. However, their work did not demonstrate a hyper-parameter search-free rule for adjusting the learning rate as a function of minibatch size, which is a central contribution of our work.

In recent work, Bottou *et al*. [4] (§4.2) review theoretical tradeoffs of minibatching and show that with the linear scaling rule, solvers follow the same training curve as a function of number of examples seen, and suggest the learning rate should not exceed a maximum rate independent of minibatch size (which justifies warmup). Our work empirically tests these theories with unprecedented minibatch sizes.

2.2. Warmup

As we discussed, for large minibatches (*e.g*., 8k) the linear scaling rule breaks down when the network is changing rapidly, which commonly occurs in early stages of training. We find that this issue can be alleviated by a properly designed *warmup* [16], namely, a strategy of using less aggressive learning rates at the start of training.

Constant warmup. The warmup strategy presented in [16] uses a low *constant* learning rate for the first few epochs of training. As we will show in §5, we have found constant warmup particularly helpful for prototyping object detection and segmentation methods [9, 31, 26, 14] that fine-tune pre-trained layers together with newly initialized layers.

In our ImageNet experiments with a large minibatch of size kn, we have tried to train with the low learning rate of η for the first 5 epochs and then return to the target learning rate of $\hat{\eta} = k\eta$. However, given a large k, we find that this constant warmup is not sufficient to solve the optimization problem, and a transition out of the low learning rate warmup phase can cause the training error to spike. This leads us to propose the following gradual warmup.

Gradual warmup. We present an alternative warmup that *gradually* ramps up the learning rate from a small to a large value. This ramp avoids a sudden increase of the learning rate, allowing healthy convergence at the start of training. In practice, with a large minibatch of size kn, we start from a learning rate of η and increment it by a constant amount at each iteration such that it reaches $\hat{\eta} = k\eta$ after 5 epochs (results are robust to the exact duration of warmup). After the warmup, we go back to the original learning rate schedule.

2.3. Batch Normalization with Large Minibatches

Batch Normalization (BN) [19] computes statistics along the minibatch dimension: this breaks the independence of each sample's loss, and changes in minibatch size change the underlying definition of the loss function being optimized. In the following we will show that a commonly used 'shortcut', which may appear to be a practical consideration to avoid communication overhead, is actually necessary for preserving the loss function when changing minibatch size.

We note that (1) and (2) assume the per-sample loss $l(x, w)$ is independent of all other samples. This is *not* the case when BN is performed and activations are computed across samples. We write $l_\mathcal{B}(x, w)$ to denote that the loss of a single sample x depends on the statistics of all samples in its minibatch $\mathcal{B}$. We denote the loss over a single minibatch $\mathcal{B}$ of size n as $L(\mathcal{B}, w) = \frac{1}{n}\sum_{x\in\mathcal{B}} l_\mathcal{B}(x, w)$. With BN, the training set can be thought of as containing all distinct subsets of size n drawn from the original training set X, which we denote as X^n. The training loss $L(w)$ then becomes:

$$L(w) = \frac{1}{|X^n|}\sum_{\mathcal{B}\in X^n} L(\mathcal{B}, w). \quad (5)$$

根据（2）的结论，在进行了多次学习率为 η、小批量大小为 n 的随机梯度下降迭代之后，我们得到：

$$w_{t+k} = w_t - \eta\frac{1}{n}\sum_{j<k}\sum_{x\in\mathcal{B}_j}\nabla l(x, w_{t+j}). \quad (3)$$

另一方面，使用大小为 kn、学习率为$\hat{\eta}$的较大小批量 $\cup_j\mathcal{B}_j$ 进行单步更新后，得到的结果为：

$$\hat{w}_{t+1} = w_t - \hat{\eta}\frac{1}{kn}\sum_{j<k}\sum_{x\in\mathcal{B}_j}\nabla l(x, w_t). \quad (4)$$

正如预期，此时的更新值会存在差异，几乎不可能出现$\hat{w}_{t+1} = w_{t+k}$的情况。不过，如果我们能够假设 $j < k$满足 $\nabla l(x, w_t) \approx \nabla l(x, w_{t+j})$，那么设定$\hat{\eta} = k\eta$ 后就能得到$\hat{w}_{t+1} \approx w_{t+k}$，此时小批量SGD与大批量SGD的更新结果将会相似。尽管这是一个较强的假设，但我们需要强调的是，只有在我们设定$\hat{\eta} = k\eta$的情况下，这两种更新结果才会相似。仅此而已。

上述解释为我们理解在何种情况下有望应用线性缩放规则提供了直观依据。在我们的实验中，采用$\hat{\eta} = k\eta$ 并加入预热阶段后，无论是小批量SGD还是大批量SGD训练出的模型，不仅最终的平均精度相同，其训练曲线也极为接近。我们的实证结果表明，在大规模的真实世界数据场景下，上述近似方法或许是有效的。

不过，至少存在两种情况明显不满足该条件$\nabla l(x、\ w_t) \approx \nabla l(x、\ w_{t+j})$。首先，在网络参数变化迅速的初始训练阶段，这一条件就不成立。为了解决这个问题，我们采用了预热阶段，相关内容详见§2.2。其次，小批量大小并非可以无限扩展：虽然在一定范围的内，不同大小带来的训练结果较为稳定，但一旦超过某个临界点，模型的准确率就会急剧下降。有趣的是，在ImageNet实验中，这个临界值恰好相当于 ~8k。

讨论部分。上述线性缩放规则最早是由Krizhevsky提出的[21]，，即便此前也有人使用过类似方法。不过Krizhevsky的研究显示，当小批量大小从128增加到1024时，模型的误差会上升1%，而我们则展示了如何在更宽范围的小批量大小下依然保持模型准确率。Chen等人也进行了相关研究。 [5]他们对比了多种分布式随机梯度下降实现方式，尽管他们的研究同样采用了线性缩放规则，但却没有建立小样本小批量基线作为参考。Li在（§4.6）章节中展示了使用容量高达5120的小批量数据进行分布式ImageNet训练的情况，且在收敛后模型准确率并未出现下降。然而他们的研究并未提出一种无需进行超参数搜索、就能根据小批量大小自动调整学习率的规则，而这正是我们工作的核心贡献之一。

在近期的研究中，Bottou等人在 [4]（§4.2)中分析了小批量训练相关的理论权衡，指出在采用线性缩放规则时，优化器的训练曲线会随所见样本数量的增加而变化，同时建议学习率不应超过一个与小批量大小无关的最大值——这一结论也为预热阶段的存在提供了依据。我们的研究则通过前所未有的大尺寸小批量数据，对这些理论进行了实证检验。

2.2. 预热阶段

正如我们所讨论的，对于大尺寸的小批量训练（例如8k批量大小），当网络处于快速变化阶段时，线性缩放规则就不再适用，而这种情况在训练初期尤为常见。我们发现，通过合理设计预热阶段 [16]，，也就是在训练初期采用较为温和的学习率，就能够有效缓解这一问题。

恒定预热策略。本文介绍的预热策略 [16]会在使用训练的最初几轮中采用较低的恒定学习率。正如我们将在§5中展示的那样，我们发现恒定预热策略对于原型设计用于共同微调预训练层与全新初始化层的物体检测及分割方法尤为有效[9, 31, 26, 14]。

在针对规模为 kn的大小批量ImageNet数据集进行的实验中，我们尝试在前5个训练轮次采用η 的较低学习率进行训练，之后再恢复到目标学习率$\hat{\eta} = k\eta$。然而，由于 k的值较大，我们发现这种恒定预热策略并不足以解决优化问题，而且一旦结束低学习率预热阶段，训练误差就可能会出现剧烈上升。正因如此，我们提出了以下渐进式预热方案。

渐进式预热策略。我们在此提出另一种预热方式，即<span style='id='3'>逐步将学习率从较小值提升至较大值。这种提升方式避免了学习率的突然增加，从而使模型在训练初期能够稳定收敛。在实际应用中，当使用规模为 kn的大小批量数据时，我们会以 η 作为初始学习率，然后在每次迭代中按固定幅度递增学习率，这样经过5个训练轮次后学习率即可达到$\hat{\eta} = k\eta$（该结果并不受预热持续时间的严格影响）。完成预热阶段后，我们会恢复原有的学习率调度方案。

2.3. 使用大批量训练的批量归一化

批量归一化（BN） [19] 会沿着小批量维度计算统计量：这一操作打破了各个样本损失值之间的独立性，同时小批量大小的改变也会影响正在优化的损失函数的定义。在下文中，我们将证明，一种看似出于减少通信开销而采用的常见‘捷径’，实际上对于在改变小批量大小时保持损失函数的稳定性而言是不可或缺的。

需要指出的是，(1)和(2)的前提是每个单个样本的$l(x, w)$损失值与所有其他样本相互独立。但在应用归一化层并跨多个样本计算激活值时，情况并非如此。我们用 $l_\mathcal{B}(x, w)$来表示某个单个样本 x 的损失值会依赖于其所在小批量 $\mathcal{B}$中所有样本的统计特征。而大小为 n 的单一小批量的损失值$\mathcal{B}$ 则可表示为 $L(\mathcal{B}, w) = \frac{1}{n}\sum_{x\in\mathcal{B}} l_\mathcal{B}(x, w)$。在采用归一化层的情况下，训练集可被视为由从原始训练集中抽取的所有大小为 n 的不同子集组成 X，我们将其记为 X^n。此时，训练损失 $L(w)$的表达式即为：

$$L(w) = \frac{1}{|X^n|}\sum_{\mathcal{B}\in X^n} L(\mathcal{B}, w). \quad (5)$$

If we view $\mathcal{B}$ as a 'single sample' in X^n, then the loss of each single sample $\mathcal{B}$ is computed *independently*.

Note that the minibatch size n over which the BN statistics are computed is a key component of the loss: if the per-worker minibatch sample size n is changed, *it changes the underlying loss function L that is optimized*. More specifically, the mean/variance statistics computed by BN with different n exhibit different levels of random variation.

In the case of distributed (and multi-GPU) training, if the per-worker sample size n is kept fixed and the total minibatch size is kn, it can be viewed a minibatch of k samples with each sample $\mathcal{B}_j$ independently selected from X^n, so the underlying loss function is unchanged and is still defined in X^n. Under this point of view, in the BN setting after seeing k minibatches $\mathcal{B}_j$, (3) and (4) become:

$$w_{t+k} = w_t - \eta \sum_{j<k} \nabla L(\mathcal{B}_j, w_{t+j}), \tag{6}$$

$$\hat{w}_{t+1} = w_t - \hat{\eta} \frac{1}{k} \sum_{j<k} \nabla L(\mathcal{B}_j, w_t). \tag{7}$$

Following similar logic as in §2.1, we set $\hat{\eta} = k\eta$ and *we keep the per-worker sample size n constant when we change the number of workers k*.

In this work, we use $n = 32$ which has performed well for a wide range of datasets and networks [19, 16]. If n is adjusted, it should be viewed as a hyper-parameter of BN, not of distributed training. We also note that *the BN statistics should* not *be computed across all workers*, not only for the sake of reducing communication, but also for maintaining the same underlying loss function being optimized.

3. Subtleties and Pitfalls of Distributed SGD

In practice a distributed implementation has many subtleties. Many common implementation errors change the definitions of hyper-parameters, leading to models that train but whose error may be higher than expected, and such issues can be difficult to discover. While the remarks below are straightforward, they are important to consider explicitly to faithfully implement the underlying solver.

Weight decay. Weight decay is actually the outcome of the gradient of an L2-regularization term *in the loss function*. More formally, the per-sample loss in (1) can be written as $l(x, w) = \frac{\lambda}{2}\|w\|^2 + \varepsilon(x, w)$. Here $\frac{\lambda}{2}\|w\|^2$ is the sample-independent L2 regularization on the weights and $\varepsilon(x, w)$ is a sample-dependent term such as the cross-entropy loss. The SGD update in (2) can be written as:

$$w_{t+1} = w_t - \eta\lambda w_t - \eta \frac{1}{n} \sum_{x \in \mathcal{B}} \nabla \varepsilon(x, w_t). \tag{8}$$

In practice, usually only the sample-dependent term $\sum \nabla \varepsilon(x, w_t)$ is computed by backprop; the term λw_t is computed *separately* and added to the aggregated gradients contributed by $\varepsilon(x, w_t)$. If there is no weight decay term, there are many equivalent ways of scaling the learning rate, including scaling the term $\varepsilon(x, w_t)$. However, as can be seen from (8), in general this is *not* the case. We summarize these observations in the following remark:

Remark 1: Scaling the cross-entropy loss is not *equivalent to scaling the learning rate.*

Momentum correction. Momentum SGD is a commonly adopted modification to the vanilla SGD in (2). A reference implementation of momentum SGD has the following form:

$$\begin{aligned} u_{t+1} &= m u_t + \frac{1}{n} \sum_{x \in \mathcal{B}} \nabla l(x, w_t) \\ w_{t+1} &= w_t - \eta u_{t+1}. \end{aligned} \tag{9}$$

Here m is the momentum decay factor and u is the update tensor. A popular variant absorbs the learning rate η into the update tensor. Substituting v_t for ηu_t in (9) yields:

$$\begin{aligned} v_{t+1} &= m v_t + \eta \frac{1}{n} \sum_{x \in \mathcal{B}} \nabla l(x, w_t) \\ w_{t+1} &= w_t - v_{t+1}. \end{aligned} \tag{10}$$

For a fixed η, the two are equivalent. However, we note that while u only depends on the gradients and is independent of η, v is entangled with η. When η changes, to maintain equivalence with the reference variant in (9), the update for v should be: $v_{t+1} = m\frac{\eta_{t+1}}{\eta_t} v_t + \eta_{t+1} \frac{1}{n} \sum \nabla l(x, w_t)$. We refer to the factor $\frac{\eta_{t+1}}{\eta_t}$ as the *momentum correction*. We found that this is especially important for stabilizing training when $\eta_{t+1} \gg \eta_t$, otherwise the history term v_t is too small which leads to instability (for $\eta_{t+1} < \eta_t$ momentum correction is less critical). This leads to our second remark:

Remark 2: Apply momentum correction after changing learning rate if using (10).

Gradient aggregation. For k workers each with a per-worker minibatch of size n, following (4), gradient aggregation must be performed over the entire set of kn examples according to $\frac{1}{kn} \sum_j \sum_{x \in \mathcal{B}_j} l(x, w_t)$. Loss layers are typically implemented to compute an average loss over their *local* input, which amounts to computing a per-worker loss of $\sum l(x, w_t)/n$. Given this, correct aggregation requires *averaging* the k gradients in order to recover the missing $1/k$ factor. However, standard communication primitives like allreduce [11] perform summing, not averaging. Therefore, it is more efficient to absorb the $1/k$ scaling into the loss, in which case only the loss's gradient with respect to its input needs to be scaled, removing the need to scale the entire gradient vector. We summarize this as follows:

Remark 3: Normalize the per-worker loss by total *minibatch size kn,* not *per-worker size n.*

We also note that it may be incorrect to 'cancel k' by setting $\hat{\eta} = \eta$ (not $k\eta$) and normalizing the loss by $1/n$ (not $1/kn$), which can lead to incorrect weight decay (see *Remark 1*).

如果将 $\mathcal{B}$ 视为 X^n中的'单个样本'，那么每个这样的单个样本 $\mathcal{B}$ 的损失值都是独立地计算得出的。

需要注意的是，用于计算均值/方差统计量的小批量大小 n 是损失值中的关键组成部分：如果每个工作节点所处理的小批量样本数量 n 发生变化，就会改变正在被优化的损失函数 L 。更具体地说，使用不同 n 参数的BN算法计算出的均值/方差统计量，其随机波动程度也会存在差异。

在分布式（以及多GPU）训练场景中，如果保持每个工作节点的样本数量 n 不变，而总小批量大小为 kn，那么就可以将其视为一组由 k 多个样本组成的小批量，且这些样本 $\mathcal{B}_j$ 都是从 X^n中独立选取的 X^n。从这个角度出发，在BN框架下，在考察了 k 小批量数据 $\mathcal{B}_j$之后，(3)和(4)这两个表达式的含义也就随之确定了。

$$w_{t+k} = w_t - \eta \sum_{j<k} \nabla L(\mathcal{B}_j, w_{t+j}), \tag{6}$$

$$\hat{w}_{t+1} = w_t - \hat{\eta} \frac{1}{k} \sum_{j<k} \nabla L(\mathcal{B}_j, w_t). \tag{7}$$

遵循与§2.1中类似的逻辑，我们在调整工作节点数量时，会保持每个工作节点的样本数量不变，并设定$\hat{\eta} = k\eta$ 为某个特定值。

在本研究中，我们采用了 $n = 32$这种在多种数据集和不同类型网络上均表现优异的算法 [19, 16]。如果 n 该参数被调整，那么它应被视为BN算法的超参数，而非分布式训练的超参数。此外还需注意，*BN*算法的统计量不应在所有工作节点上同时计算，这样做不仅有助于减少通信量，还能确保优化过程中所使用的损失函数保持一致。

3. 分布式随机梯度下降的细微问题与隐患

在实际应用中，分布式实现的细节非常多。许多常见的实现错误会改变超参数的定义，从而导致模型虽然能够训练成功，但其误差却可能高于预期，而这类问题往往很难被发现。尽管以下的说明较为简单，但在准确实现相关求解器时，仍需明确加以考虑。

权重衰减。实际上，权重衰减就是损失函数中*L2*正则化项的梯度所产生的效应。更正式地来说，(1)式中所示的每个样本对应的损失值可以表示为$l(x,\ w) = \frac{\lambda}{2}\|w\|^2 + \varepsilon(x,\ w)$。此处 $\frac{\lambda}{2}\|w\|^2$代表对权重施加的与样本无关的L2正则化处理，而 $\varepsilon(x,\ w)$则是诸如交叉熵损失之类的与样本相关的项。公式(2)中的随机梯度下降更新规则可表示为：

$$w_{t+1} = w_t - \eta\lambda w_t - \eta \frac{1}{n} \sum_{x \in \mathcal{B}} \nabla \varepsilon(x, w_t). \tag{8}$$

在实际应用中，通常只有与样本相关的项$\sum \nabla \varepsilon(x,\ w_t)$会通过反向传播来计算；而项 λw_t则会被单独计算出来，之后再加入这些汇总后的梯度中。

由 $\varepsilon(x,\ w_t)$贡献。如果不存在权重衰减项，其实有许多等效的方法可以对学习率进行缩放，其中包括对项 $\varepsilon(x,\ w_t)$ 进行缩放。不过，如(8)所示，一般情况下情况并非如此。我们在下文的说明中对这些观察结果进行了总结：

备注*1*：调整交叉熵损失值并不等同于调整学习率。

动量修正。在（2）中，动量随机梯度下降是普通SGD的一种常见改进形式。动量随机梯度下降的实现方式如下：

$$\begin{aligned} u_{t+1} &= m u_t + \frac{1}{n} \sum_{x \in \mathcal{B}} \nabla l(x, w_t) \\ w_{t+1} &= w_t - \eta u_{t+1}. \end{aligned} \tag{9}$$

此处 m 为动量衰减因子，而 u 则为更新张量。有一种常见的变体会将学习率 η 整合到更新张量中。将 v_t 代入（9）中的 ηu_t 后，可得：

$$\begin{aligned} v_{t+1} &= m v_t + \eta \frac{1}{n} \sum_{x \in \mathcal{B}} \nabla l(x, w_t) \\ w_{t+1} &= w_t - v_{t+1}. \end{aligned} \tag{10}$$

当固定的 η值相同时，这两种方法的效果是等价的。不过需要注意的是，虽然 u 仅取决于梯度且与 η无关， v 却与 η存在关联。当 η 发生变化时，为了与（9）中的参考实现保持等价，v的更新公式应为： $v_{t+1} = m\frac{\eta_{t+1}}{\eta_t} v_t + \eta_{t+1} \frac{1}{n} \sum \nabla l(x,\ w_t)$。我们将该系数称为 $\frac{\eta_{t+1}}{\eta_t}$ 动量修正。我们发现，在 $\eta_{t+1} \gg \eta_t$的情况下，这一修正对于稳定训练尤为重要，否则历史项 v_t 的值会过小，从而导致训练不稳定（因为在 $\eta_{t+1} < \eta_t$ 情况下动量修正的重要性较低）。这也引出了我们的第二点备注：

备注*2*：若采用（*10*）中的方法，在调整学习率后需进行动量修正。

梯度聚合。 对于 k 个各拥有大小为 n的单独小批量的工作节点 kn，需依据$\frac{1}{kn} \sum_j \sum_{x \in \mathcal{B}_j} l\ (x,\ w_t)$ 的要求，对所有 kn 样本执行全量归约操作。通常，损失层会被设计用来计算其局部输入对应的平均损失，这实际上等同于计算每个工作节点的损失值$\sum l\ (x,\ w_t)\ /n$。基于此，要实现正确的聚合，就需要对这些梯度进行平均处理，从而补回缺失的 $1/k$系数。不过，像全量归约操作这类标准的通信机制是执行求和而非求平均操作。因此，将比例因子直接纳入损失函数中更为高效，这样一来只需对损失函数与其输入之间的梯度进行缩放，便无需对整个梯度向量都进行缩放。我们将其总结如下：

备注*3*：应对单个工作节点的损失值进行归一化处理，归一化的分母应为整个小批量大小 kn，而非单个工作节点的大小 n。

此外还需注意的是，通过设置$\hat{\eta} = \eta$ （而非 $k\eta$）来 '取消 k' ，并使用 $1/n$ （而非 $1/kn$）对损失值进行归一化，这种做法可能是不正确的，因为它会导致权重衰减出现偏差（详见备注*1*）。

Data shuffling. SGD is typically analyzed as a process that samples data randomly *with replacement*. In practice, common SGD implementations apply *random shuffling* of the training set during each SGD epoch, which can give better results [3, 13]. To provide fair comparisons with baselines that use shuffling (*e.g.*, [16]), we ensure the samples in one epoch done by k workers are from a single consistent random shuffling of the training set. To achieve this, for each epoch we use a random shuffling that is partitioned into k parts, each of which is processed by one of the k workers. Failing to correctly implement random shuffling in multiple workers may lead to noticeably different behavior, which may contaminate results and conclusions. In summary:

Remark 4: Use a single random shuffling of the training data (per epoch) that is divided amongst all k workers.

4. Communication

In order to scale beyond the 8 GPUs in a single Big Basin server [24], gradient aggregation has to span across servers on a network. To allow for near perfect linear scaling, the aggregation must be performed *in parallel* with backprop. This is possible because there is no data dependency between gradients across layers. Therefore, as soon as the gradient for a layer is computed, it is aggregated across workers, while gradient computation for the next layer continues (as discussed in [5]). We give full details next.

4.1. Gradient Aggregation

For every gradient, aggregation is done using an *allreduce* operation (similar to the MPI collective operation *MPI_Allreduce* [11]). Before allreduce starts every GPU has its locally computed gradients and after allreduce completes every GPU has the sum of all k gradients. As the number of parameters grows and compute performance of GPUs increases, it becomes harder to hide the cost of aggregation in the backprop phase. Training techniques to overcome these effects are beyond the scope of this work (*e.g.*, quantized gradients [18], Block-Momentum SGD [6]). However, at the scale of this work, collective communication was not a bottleneck, as we were able to achieve near-linear SGD scaling by using an optimized allreduce implementation.

Our implementation of allreduce consists of three phases for communication within and across servers: (1) buffers from the 8 GPUs within a server are summed into a single buffer for each server, (2) the results buffers are shared and summed across all servers, and finally (3) the results are broadcast onto each GPU. For the local reduction and broadcast in phases (1) and (3) we used NVIDIA Collective Communication Library (NCCL)[3] for buffers of size 256 KB or more and a simple implementation consisting of a number of GPU-to-host memory copies and a CPU reduction otherwise. NCCL uses GPU kernels to accelerate intraserver collectives, so this approach dedicates more time on the GPU to backprop while using the CPU resources that would otherwise have been idle to improve throughput.

For interserver allreduce, we implemented two of the best algorithms for bandwidth-limited scenarios: the *recursive halving and doubling algorithm* [30, 37] and the *bucket algorithm* (also known as the ring algorithm) [2]. For both, each server sends and receives $2\frac{p-1}{p}b$ bytes of data, where b is the buffer size in bytes and p is the number of servers. While the halving/doubling algorithm consists of $2\log_2(p)$ communication steps, the ring algorithm consists of $2(p-1)$ steps. This generally makes the halving/doubling algorithm faster in latency-limited scenarios (*i.e.*, for small buffer sizes and/or large server counts). In practice, we found the halving/doubling algorithm to perform much better than the ring algorithm for buffer sizes up to a million elements (and even higher on large server counts). On 32 servers (256 GPUs), using halving/doubling led to a speedup of 3× over the ring algorithm.

The halving/doubling algorithm consists of a reduce-scatter collective followed by an allgather. In the first step of reduce-scatter, servers communicate in pairs (rank 0 with 1, 2 with 3, *etc.*), sending and receiving for different halves of their input buffers. For example, rank 0 sends the second half of its buffer to 1 and receives the first half of the buffer from 1. A reduction over the received data is performed before proceeding to the next step, where the distance to the destination rank is doubled while the data sent and received is halved. After the reduce-scatter phase is finished, each server has a portion of the final reduced vector.

This is followed by the allgather phase, which retraces the communication pattern from the reduce-scatter in reverse, this time simply concatenating portions of the final reduced vector. At each server, the portion of the buffer that was being sent in the reduce-scatter is received in the allgather, and the portion that was being received is now sent.

To support non-power-of-two number of servers, we used the *binary blocks algorithm* [30]. This is a generalized version of the halving/doubling algorithm where servers are partitioned into power-of-two blocks and two additional communication steps are used, one immediately after the intrablock reduce-scatter and one before the intrablock allgather. Non-power-of-two cases have some degree of load imbalance compared to power-of-two, though in our runs we did not see significant performance degradation.

4.2. Software

The allreduce algorithms described are implemented in *Gloo*[4], a library for collective communication. It supports

[3] https://developer.nvidia.com/nccl

[4] https://github.com/facebookincubator/gloo

数据打乱。随机梯度下降通常被视为一项会随机抽取数据并允许重复抽取的过程。在实际应用中，常见的随机梯度下降实现会在每一轮训练中对训练集进行随机打乱，这样做往往能获得更好的训练效果 [3, 13]。为了与那些采用数据打乱技术的基线方法进行公平对比（例如， [16]），我们会确保由 k 工作节点在单轮训练中生成的样本都来自对训练集进行的同一套随机打乱操作。为此，我们在每一轮训练中都会将数据集划分成 k多个部分，每个部分由一个 k 工作节点负责处理。如果多个工作节点未能正确实现数据随机打乱，就可能会导致行为出现显著差异，进而影响最终的结果与结论。综上所述：

备注*4*：对训练数据进行一次随机打乱操作（每个训练轮次一次），然后将这些数据分配给所有的 k 工作节点。

4. 通信机制

若要突破单台Big Basin服务器上8块GPU的限制实现更大规模的训练，就必须通过网络在多台服务器之间开展 [24]，梯度聚合。为达成近乎完美的线性扩展效果，该聚合操作必须与反向传播并行执行。之所以能够做到这一点，是因为不同层之间的梯度之间不存在数据依赖关系。因此，一旦某层的梯度计算完成，就会立即在所有工作节点间进行聚合，与此同时下一层的梯度计算也会继续进行（具体内容见 [5]）。更多详细信息将在后文阐述。

4.1. 梯度聚合

对于每一个梯度，都是通过全量归约操作来进行聚合的（该操作与MPI集合操作*MPI Allreduce* 类似 [11]）。在全量归约开始之前，每块GPU都拥有自己本地计算出的梯度；而当全量归约完成后，每块GPU都会得到所有 k 梯度之和。随着参数数量的增加以及GPU计算性能的提升，在反向传播阶段掩盖聚合操作所带来的开销变得越来越困难。用于克服这些问题的训练技术已超出本研究的范畴（例如量化梯度 [18]，、Block-Momentum SGD [6]等）。不过，在本研究所采用的规模下，由于采用了优化过的全量归约实现方式，集合通信并未成为瓶颈，我们依然实现了近乎线性的随机梯度下降扩展效果。

我们实现的全量归约操作包含三个阶段，用于处理服务器内部以及服务器之间的数据通信：首先，汇总每台服务器内8个图形处理器中的缓冲区内容，为每台服务器生成一个单一的缓冲区；其次，在所有服务器之间共享这些结果缓冲区并再次进行求和；最后，将最终结果广播到每个图形处理器上。在阶段（1）的本地归约操作以及阶段（3）的广播操作中，我们采用了NVIDIA集体通信库（NCCL）[3] 来处理大小为256 KB及以上的缓冲区，同时也采用了另一种由简单组件构成的实现方式否则则会增加图形处理器与主机之间的内存复制次数，并依赖CPU来进行归约运算。由于NCCL利用图形处理器内核来加速服务器内部的集体通信操作，因此这种方案能够将更多图形处理器的计算时间用于反向传播过程，同时利用原本处于空闲状态的CPU资源来提升整体吞吐量。</style>

针对服务器间全量归约操作，我们为延迟受限场景实现了两种最优算法：递归减半加倍算法 [30, 37]以及桶算法（亦称环算法） [2]。对于这两种算法，每台服务器都需要发送和接收 $2\frac{p-1}{p}b$ 一定字节量的数据，其中 b 表示以字节为单位的缓冲区大小， p 则表示服务器数量。减半加倍算法包含$2\log_2$（p）个通信步数，而环算法则包含$2(p-1)$ 个步数。通常在延迟受限场景下，这种结构使得减半加倍算法的运行速度更快（即，在缓冲区大小较小或服务器数量较多时）。实际测试表明，对于缓冲区大小高达百万个元素的情况（甚至在服务器数量更多时），减半加倍算法的性能远优于环算法。在拥有32台服务器（共256块图形处理器）的测试环境中，采用减半加倍算法相比环算法可实现 3× 更高的加速比。

减半加倍算法由归约散射集体操作和全收集操作组成。在归约散射操作的第一步中，各服务器以成对的方式相互通信（即等级0与等级1通信、等级2与等级3通信，以此类推），分别发送和接收自身输入缓冲区中对应的一半数据。例如，等级0会将其缓冲区的后半部分发送给等级1，同时从等级1那里接收该缓冲区的前半部分。在进入下一步之前，首先会对已接收到的数据进行归约处理；接着，发送和接收数据的距离会翻倍，而数据量则减半。当归约散射阶段结束后，每台服务器都会持有最终归约向量的相应部分。

紧接着是全收集操作阶段，该阶段会反向重复归约散射时的通信模式，只不过此时只是将最终归约向量的各个部分直接拼接起来。在每台服务器上，其在归约散射阶段发送出去的缓冲区部分会在全收集阶段被接收回来，而此前接收到的部分则会被发送出去。

为支持服务器数量并非2的幂的情况，我们采用了二进制块算法 [30]。该算法是减半加倍算法的扩展版本，它将服务器划分为2的幂大小的块，并额外增加两步通信步数：一步在块内归约散射操作之后立即执行，另一步则在块内全收集操作之前执行。与2的幂情况相比，非2的幂场景会存在一定程度的负载不平衡，不过在我们的测试中并未观察到明显的性能下降。

4.2. 软件

文中所述的各全量归约操作算法均实现于*Gloo*[4], 这一用于集体通信的库中。该库支持

[3] https://developer.nvidia.com/nccl

[4] https://github.com/facebookincubator/gloo

multiple communication contexts, which means no additional synchronization is needed to execute multiple allreduce instances in parallel. Local reduction and broadcast (described as phases (1) and (3)) are pipelined with inter-server allreduce where possible.

Caffe2 supports multi-threaded execution of the compute graph that represents a training iteration. Whenever there is no data dependency between subgraphs, multiple threads can execute those subgraphs in parallel. Applying this to backprop, local gradients can be computed in sequence, without dealing with allreduce or weight updates. This means that during backprop, the set of *runnable subgraphs* may grow faster than we can execute them. For subgraphs that contain an allreduce run, all servers must choose to execute the same subgraph from the set of runnable subgraphs. Otherwise, we risk distributed deadlock where servers are attempting to execute non-intersecting sets of subgraphs. With allreduce being a *collective* operation, servers would time out waiting. To ensure correct execution we impose a partial order on these subgraphs. This is implemented using a cyclical control input, where completion of the n-th allreduce unblocks execution of the $(n+c)$-th allreduce, with c being the maximum number of concurrent allreduce runs. Note that this number should be chosen to be lower than the number of threads used to execute the full compute graph.

4.3. Hardware

We used Facebook's Big Basin [24] GPU servers for our experiments. Each server contains 8 NVIDIA Tesla P100 GPUs that are interconnected with NVIDIA NVLink. For local storage, each server has 3.2TB of NVMe SSDs. For network connectivity, the servers have a Mellanox ConnectX-4 50Gbit Ethernet network card and are connected to Wedge100 [1] Ethernet switches.

We have found 50Gbit of network bandwidth sufficient for distributed synchronous SGD for ResNet-50, per the following analysis. ResNet-50 has approximately 25 million parameters. This means the total size of parameters is $25 \cdot 10^6 \cdot \text{sizeof(float)} = 100\text{MB}$. Backprop for ResNet-50 on a single NVIDIA Tesla P100 GPU takes 120 ms. Given that allreduce requires $\sim 2\times$ bytes on the network compared to the value it operates on, this leads to a peak bandwidth requirement of $200\text{MB}/0.125\text{s} = 1600\text{MB/s}$, or 12.8 Gbit/s, not taking into account communication overhead. When we add a smudge factor for network overhead, we reach a peak bandwidth requirement for ResNet-50 of ~15 Gbit/s.

As this peak bandwidth requirement only holds during backprop, the network is free to be used for different tasks that are less latency sensitive then aggregation (*e.g.* reading data or saving network snapshots) during the forward pass.

5. Main Results and Analysis

Our main result is that we can train ResNet-50 [16] on ImageNet [33] using 256 workers in one hour, while matching the accuracy of small minibatch training. Applying the linear scaling rule along with a warmup strategy allows us to seamlessly scale between small and large minibatches (up to 8k images) without tuning additional hyper-parameters or impacting accuracy. In the following subsections we: (1) describe experimental settings, (2) establish the effectiveness of large minibatch training, (3) perform a deeper experimental analysis, (4) show our findings generalize to object detection/segmentation, and (5) provide timings.

5.1. Experimental Settings

The 1000-way ImageNet classification task [33] serves as our main experimental benchmark. Models are trained on the ~1.28 million training images and evaluated by top-1 error on the 50,000 validation images.

We use the ResNet-50 [16] variant from [12], noting that the stride-2 convolutions are on 3×3 layers instead of on 1×1 layers as in [16]. We use Nesterov momentum [29] with m of 0.9 following [12] but note that standard momentum as was used in [16] is equally effective. We use a weight decay λ of 0.0001 and following [16] we do not apply weight decay on the learnable BN coefficients (namely, γ and β in [19]). In order to keep the training objective fixed, which depends on the BN batch size n as described in §2.3, we use $n = 32$ throughout, regardless of the overall minibatch size. As in [12], we compute the BN statistics using running average (with momentum 0.9).

All models are trained for 90 epochs regardless of minibatch sizes. We apply the *linear scaling rule* from §2.1 and use a learning rate of $\eta = 0.1 \cdot \frac{kn}{256}$ that is linear in the minibatch size kn. With $k = 8$ workers (GPUs) and $n = 32$ samples per worker, $\eta = 0.1$ as in [16]. We call this number $(0.1 \cdot \frac{kn}{256})$ the *reference learning rate*, and reduce it by $1/10$ at the 30-th, 60-th, and 80-th epoch, similar to [16].

We adopt the initialization of [15] for all convolutional layers. The 1000-way fully-connected layer is initialized by drawing weights from a zero-mean Gaussian with standard deviation of 0.01. We have found that although SGD with a small minibatch is not sensitive to initialization due to BN, this is not the case for a substantially large minibatch. Additionally we require an appropriate warmup strategy to avoid optimization difficulties in early training.

For BN layers, the learnable scaling coefficient γ is initialized to be 1, *except for each residual block's last BN where γ is initialized to be 0.* Setting $\gamma = 0$ in the last BN of each residual block causes the forward/backward signal initially to propagate through the identity shortcut of ResNets, which we found to ease optimization at the start of training. This initialization improves all models but is particularly helpful for large minibatch training as we will show.

多种通信场景，因此无需额外的同步机制即可并行执行多个全量归约操作实例。在可能的情况下，本地归约与广播操作（对应阶段（1）和（3））会与服务器间的全量归约操作依次串行处理。

Caffe2 支持对代表训练迭代的计算图进行多线程执行。当各个子图之间不存在数据依赖关系时，多个线程可以并行处理这些子图。将这一机制应用于反向传播过程，即可依次计算出各局部梯度，而无需处理全量归约操作或权重更新。这意味着在反向传播过程中，可执行子图的数量增长速度可能会快于我们处理它们的能力。对于包含全量归约操作的子图，所有服务器必须选择从这些可执行子图中执行相同的子图；否则就有可能出现分布式死锁，因为各服务器会试图执行互不重叠的子图集合。由于全量归约操作属于集体性操作，服务器们会因等待而过时。为确保正确执行，我们会对这些子图设定一种偏序关系，具体是通过循环控制输入来实现——即第 n次全量归约完成之后，第$(n+c)$次全量归约才能开始执行，其中c 表示允许同时进行的最大全量归约操作次数。需要注意的是，这个数值应当设置得小于用于执行整个计算图的线程数量。

4.3. 硬件

我们在实验中使用了Facebook的Big Basin [24] 图形处理器服务器。每台服务器配备了8块NVIDIA Tesla P100显卡，这些显卡通过NVIDIA NVLink技术相互连接。在本地存储方面，每台服务器拥有3.2TB容量的NVMe固态硬盘。至于网络连接，这些服务器配备了Mellanox ConnectX-4 50Gbit以太网网卡，并与Wedge100 [1] 以太网交换机相连。</

通过以下分析，我们发现50Gbit的网络带宽足以支持针对ResNet-50模型的分布式同步随机梯度下降训练。ResNet-50大约包含2500万个参数，因此所有参数的总大小为$25 \cdot 10^6 \cdot \text{sizeof(float)} = 100\text{MB}$。在单块NVIDIA Tesla P100显卡上对ResNet-50进行反向传播运算需要120毫秒。由于全量归约操作需要在网络上传输 ~2× 字节的数据，相较于其处理的数据量而言，这会导致最高的带宽需求达到$200\text{MB}/0.125\text{s} = 1600\text{MB/s}$，即12.8 Gbit/s，此数值尚未考虑通信开销。若再加入用于弥补网络开销的缓冲系数，ResNet-50的最终最高带宽需求则为 ~15 Gbit/s。</

由于这一峰值带宽需求仅在反向传播过程中出现，因此在前向传播阶段，该网络可以用于那些对延迟要求较低的其它任务（例如读取数据或保存网络快照）。

5. 主要结果与分析

我们的主要研究结果表明，仅需256个计算单元，我们便能在1小时内完成在ImageNet [33] 数据集上对ResNet-50 [16] 网络的训练，且其准确率可与小批量训练相媲美。通过应用线性缩放规则并结合预热策略，我们无需调整额外的超参数或影响模型准确率，即可实现从小批量到大批量训练（最多支持8k张图像）的平滑过渡。在接下来的各小节中，我们将：(1)介绍实验设置，(2)验证大批量训练的有效性，(3)进行更深入的实验分析，(4)证明我们的研究结果同样适用于物体检测/分割任务，以及(5)展示相关耗时数据。

5.1. 实验设置

我们以包含1000个类别的ImageNet数据集 [33] 作为主要的实验基准。模型在 ~1280万张训练图像上进行训练，并通过50,000张验证图像上的Top-1误差来评估性能。

我们采用ResNet-50 [16] 版本， [12], 需注意这里的卷积操作是在 3×3层上而非1×1层上， [16]。我们使用Nesterov动量 [29], 其参数值为 m 0.9，这一设置沿用了 [12] 之前的方法，但也要指出， [16] 传统动量同样具有很好的效果。此外我们还设置了权重衰减 λ，其值为0.0001；同时 [16], 我们对可学习BN系数不应用权重衰减（即γ 和 β 在 [19]中的那些系数）。为确保训练目标保持不变，而该目标会随BN小批量大小 n 的变化而改变，具体说明见 § 2.3，因此无论整体小批量大小如何，我们始终使用 $n = 32$这一方法。与 [12], 之前的研究类似，我们在这里也是通过滑动平均（配合0.9的动量值）来计算BN统计量的。

无论小批量大小如何，所有模型均需训练90个训练轮次。我们采用线性缩放规则（出自 § 2.1），并使用与小批量大小成线性关系的 $\eta = 0.1 \cdot \frac{kn}{256}$ 学习率 kn。在此过程中， $k = 8$工作节点（即图形处理器）的数量为 $\eta = 0.1$，具体方式与 [16]中所述一致。我们将这个数值（$0.1 \cdot \frac{kn}{256}$）称为参考学习率，并在第30、60和80个训练轮次时按相应比例对其进行降低，其方式类似 [16]。

对于所有的卷积层，我们都采用 [15] 作为初始化方式。而那1000维的全连接层则通过从均值为0、标准差为0.01的高斯分布中抽取权重来进行初始化。我们发现，由于批归一化的存在，使用小批量进行随机梯度下降时对初始化的敏感度较低，但当小批量规模大幅增大时情况就不再如此了。此外，为了防止在训练初期出现优化难题，我们还需要采用合适的预热策略。

对于批量归一化层，其可学习缩放系数 γ 会被初始化为1，但每个残差块最后的批量归一化层除外，该层的 γ 初始化值为*0*。将每个残差块最后一个批量归一化层中的 $\gamma = 0$设为特定值，可使前向/反向信号最初通过ResNet系列模型的恒等映射路径传递，我们发现这有助于在训练初期简化优化过程。这种初始化方式能提升所有模型的性能，尤其在我们后续将展示的较大小批量训练场景中更为有效。

We use scale and aspect ratio data augmentation [36] as in [12]. The network input image is a 224×224 pixel random crop from an augmented image or its horizontal flip. The input image is normalized by the per-color mean and standard deviation, as in [12].

Handling random variation. As models are subject to random variation in training, we compute a model's error rate as the *median* error of the final 5 epochs. Moreover, we report the mean and standard deviation (std) of the error from *5 independent runs*. This gives us more confidence in our results and also provides a measure of model stability.

The random variation of ImageNet models has generally not been reported in previous work (largely due to resource limitations). We emphasize that ignoring random variation may cause unreliable conclusions, especially if results are from a single trial, or the best of many.

Baseline. Under these settings, we establish a ResNet-50 baseline using $k = 8$ (8 GPUs in one server) and $n = 32$ images per worker (minibatch size of $kn = 256$), as in [16]. Our baseline has a top-1 validation error of 23.60% ±0.12. As a reference, ResNet-50 from `fb.resnet.torch` [12] has 24.01% error, and that of the original ResNet paper [16] has 24.7% under weaker data augmentation.

5.2. Optimization or Generalization Issues?

We establish our main results on large minibatch training by exploring optimization and generalization behaviors. We will demonstrate that with a proper warmup strategy, large minibatch SGD can both match the *training curves* of small minibatch SGD and also match the *validation* error. In other words, in our experiments both *optimization* and *generalization* of large minibatch training matches that of small minibatch training. Moreover, in §5.4 we will show that these models exhibit good generalization behavior to the object detection/segmentation transfer tasks, matching the transfer quality of small minibatch models.

For the following results, we use $k = 256$ and $n = 32$, which results in a minibatch size $kn = 8$k (we use '1k' to denote 1024). As discussed, our baseline has a minibatch size of $kn = 256$ and a reference learning rate of $\eta = 0.1$. Applying the linear scaling rule gives $\eta = 3.2$ as the reference learning rate for our large minibatch runs. We test three warmup strategies as discussed in §2.2: *no warmup*, *constant warmup* with $\eta = 0.1$ for 5 epochs, and *gradual warmup* which starts with $\eta = 0.1$ and is linearly increased to $\eta = 3.2$ over 5 epochs. All models are trained from scratch and all other hyper-parameters are kept fixed. We emphasize that while better results for any particular minibatch size could be obtained by optimizing hyper-parameters for that case; *our goal is to match errors across minibatch sizes by using a general strategy that avoids hyper-parameter tuning for each minibatch size*.

	k	n	kn	η	top-1 error (%)
baseline (single server)	8	32	256	0.1	23.60 ±0.12
no warmup, Figure 2a	256	32	8k	3.2	24.84 ±0.37
constant warmup, Figure 2b	256	32	8k	3.2	25.88 ±0.56
gradual warmup, Figure 2c	256	32	8k	3.2	23.74 ±0.09

Table 1. **Validation error on ImageNet using ResNet-50** (mean and std computed over 5 trials). We compare the small minibatch model (kn=256) with large minibatch models (kn=8k) with various warmup strategies. Observe that the top-1 validation error for small and large minibatch training (with gradual warmup) is quite close: 23.60% ±0.12 *vs*. 23.74% ±0.09, respectively.

Training error. Training curves are shown in Figure 2. With no warmup (2a), the training curve for large minibatch of $kn = 8$k is inferior to training with a small minibatch of $kn = 256$ across all epochs. A constant warmup strategy (2b) actually degrades results: although the small constant learning rate can decrease error during warmup, the error spikes immediately after and training never fully recovers.

Our main result is that with gradual warmup, large minibatch training error matches the baseline training curve obtained with small minibatches, see Figure 2c. Although the large minibatch curve starts higher due to the low η in the warmup phase, it catches up shortly thereafter. After about 20 epochs, the small and large minibatch training curves match closely. The comparison between no warmup and gradual warmup suggests that *large minibatch sizes are challenged by optimization difficulties in early training* and if these difficulties are addressed, the training error and its curve can match a small minibatch baseline closely.

Validation error. Table 1 shows the *validation error* for the three warmup strategies. The no-warmup variant has ~1.2% higher validation error than the baseline which is likely caused by the ~2.1% increase in training error (Figure 2a), rather than overfitting or other causes for poor generalization. This argument is further supported by our gradual warmup experiment. The gradual warmup variant has a *validation* error within 0.14% of the baseline (noting that std of these estimates is ~0.1%). Given that the final training errors (Figure 2c) match nicely in this case, it shows that *if the optimization issues are addressed, there is no apparent generalization degradation observed using large minibatch training*, even if the minibatch size goes from 256 to 8k.

Finally, Figure 4 shows both the training and validation curves for the large minibatch training with gradual warmup. As can be seen, validation error starts to match the baseline closely after the second learning rate drop; actually, the validation curves can match earlier if BN statistics are recomputed prior to evaluating the error instead of using the running average (see also caption in Figure 4).

我们会采用尺度与宽高比数据增强 [36]，具体方式如下 [12]：网络输入图像来自经过增强处理的图像或其水平翻转后的图像，所选取的是 224×224 像素级的随机裁剪区域。随后会按照 [12] 中的方法，利用各颜色均值与标准差对输入图像进行归一化处理。

处理随机波动问题。由于模型在训练过程中会存在随机波动，我们通过计算最后5个训练轮次误差的中位数来作为模型的错误率。此外，我们还统计了*5*次独立实验所得误差的均值与标准差，这样的处理方式不仅能提升我们对结果的置信度，还能反映模型的稳定性。

以往的研究通常并未报告ImageNet模型的随机波动情况（这主要是因为存在资源限制）。我们需要强调的是，如果忽视这种随机波动，很可能会导致不可靠的结论，尤其是当结果仅来自单次实验或是在多次实验中选取的最佳结果时。

基准模型。在上述设置下，我们参照 [16]中的方法，使用 $k = 8$（即每台服务器配备8块图形处理器）以及每个工作节点处理 $n = 32$张图像（小批量大小为 $kn = 256$）来构建ResNet-50基准模型。该基准模型的Top-1验证错误率为23.60% ±0.12。作为参考，`fb.resnet.torch` [12]实现的ResNet-50模型错误率为24.01%，而基于原始ResNet论文 [16]的方法在数据增强程度较低的情况下，其错误率则为24.7%。

5.2. 优化或泛化问题?

我们通过研究优化与泛化行为，基于大批量SGD算法展开了相关实验，并得出了主要研究结果。我们将证明，只要采用恰当的预热策略，大批量随机梯度下降不仅能够与小批量SGD的训练曲线相吻合，其验证误差也能与之相当。换言之，在我们的实验中，大批量训练的优化表现以及泛化行为都与小批量训练的表现一致。此外，在§5.4部分，我们还将展示这些模型在目标检测/分割迁移任务上具备良好的泛化能力，其迁移质量可媲美小批量训练的模型。训练曲线验证误差

在以下结果中，我们采用了 $k = 256$ 与 $n = 32$，由此得出的小批量大小为 $kn = 8$k（此处用 '1k' 表示1024）。如前文所述，我们的基准方案所采用的小批量大小为 $kn = 256$，参考学习率则为$\eta = 0.1$。根据线性缩放规则，我们最终确定了用于大批量训练的参考学习率值为 $\eta = 3.2$。如§2.2中所讨论的，我们测试了三种预热策略：无预热、恒定预热策略，其预热持续时间为 $\eta = 0.1$，共5个训练轮次；以及渐进式预热策略，该策略从 $\eta = 0.1$ 开始，在5个训练轮次内逐渐提升至 $\eta = 3.2$。所有模型均从零开始训练，且其他所有超参数都保持不变。需要强调的是，虽然通过针对特定小批量大小优化超参数，或许可以获得更好的结果；但我们的目标是通过一种通用策略来消除不同小批量大小带来的误差差异，从而避免为每个批量大小单独调整超参数。

	k	n	kn	η	Top-1误差（%）
基准曲线（单台服务器）	8	32	256	0.1	23.60 ±0.12
无预热阶段，见图2a	256	32	8k	3.2	24.84 ±0.37
恒定预热策略，图 2b	256	32	8k	3.2	25.88 ±0.56
渐进式预热策略，图 2c	256	32	8k	3.2	23.74 ±0.09

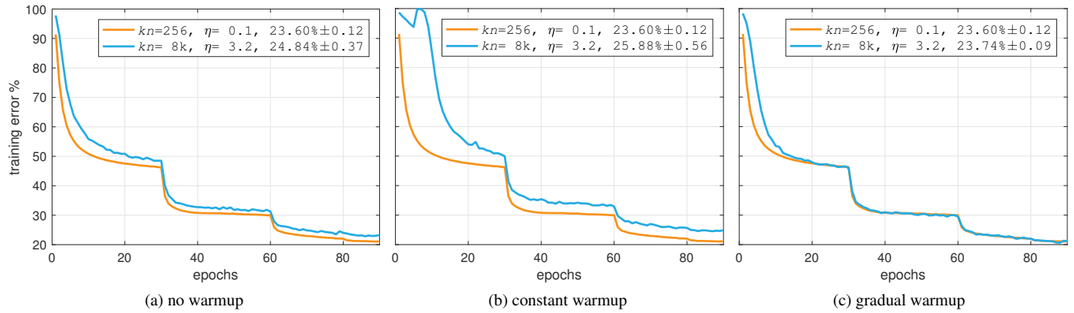
表1. **使用ResNet-50在ImageNet数据集上测试所得的验证误差**（均值与标准差为5次实验的结果）。本表对比了采用不同预热策略的小批量模型（kn=256）与大批量模型（kn=8k）。可见，在采用渐进式预热策略的情况下，小批量训练与大批量训练对应的Top-1验证错误率非常接近：分别为23.60% ±0.12 与 23.74% ±0.09。

训练误差。训练曲线如图 2所示。在采用无预热方案（2a）的情况下，大小为 $kn = 8$k的大批量训练的曲线，在所有训练轮次中的表现都要逊于使用小批量$kn = 256$ 进行的训练。而恒定预热策略（2b）实际上还会使性能下降：尽管较小的恒定学习率能够在预热阶段降低误差，但误差会在之后立即反弹，且训练过程再也无法完全恢复到初始水平。

我们的主要研究结果表明，在采用渐进式预热策略后，大尺寸小批量训练所产生的误差与使用小样本小批量训练所得到的基准曲线相当，详见图表2c。虽然由于预热阶段的较低，大尺寸小批量的误差曲线起始值会更高，但很快就能追上后者。大约经过20个训练轮次后，两种小批量训练方式的误差曲线便变得极为接近。对比未进行预热与采用渐进式预热的情况可以发现，大尺寸小批量在训练初期会面临优化方面的难题，而一旦解决了这些难题，其训练误差及其对应的误差曲线便能与小样本小批量基线曲线高度吻合。

验证误差。表1展示了三种预热策略对应的验证误差数值。未采用预热策略的模型，其验证误差比基准曲线高出0.2%，这一差异很可能是由训练误差的上升（见图表2a）所导致的，而非过拟合或其他导致泛化能力下降的因素。我们的渐进式预热实验进一步支持了这一观点：采用渐进式预热的模型，其验证误差仅比基准曲线高0.14%左右（需注意，这些数值的标准差为0.1%2c）。由于在此情况下两者的最终训练误差数值十分接近，这说明，只要解决了优化问题，即便将小批量大小从*256*提升至*8k*，使用大尺寸小批量进行训练也不会出现明显的泛化能力下降现象。

最后，图4展示了采用渐进式预热的大小批量训练方式的训练曲线与验证曲线。如图所示，在第二次学习率下调之后，验证误差开始逐渐接近基准曲线；实际上，如果在计算误差之前重新计算BN统计量而非使用滑动平均值，验证曲线甚至可以更早地与基准曲线重合（详见图4中的说明）。



(a) no warmup (b) constant warmup (c) gradual warmup

Figure 2. **Warmup.** Training error curves for minibatch size 8192 using various warmup strategies compared to minibatch size 256. *Validation* error (mean±std of 5 runs) is shown in the legend, along with minibatch size kn and reference learning rate η.

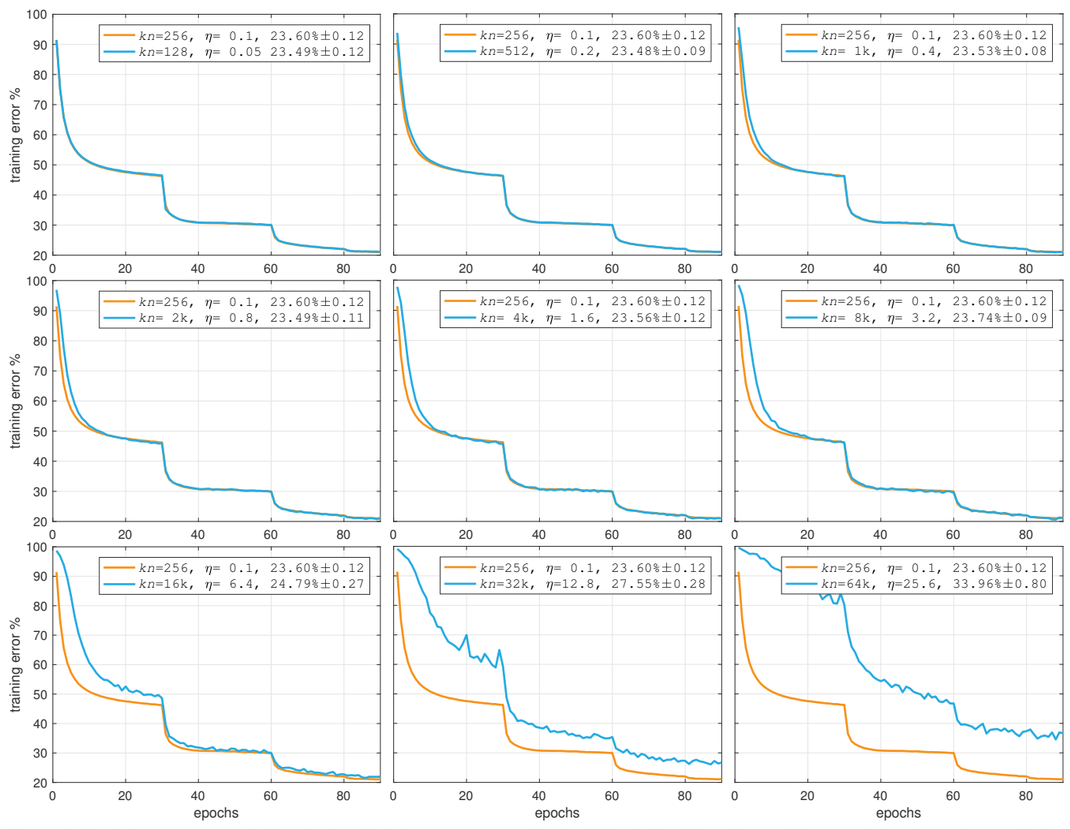
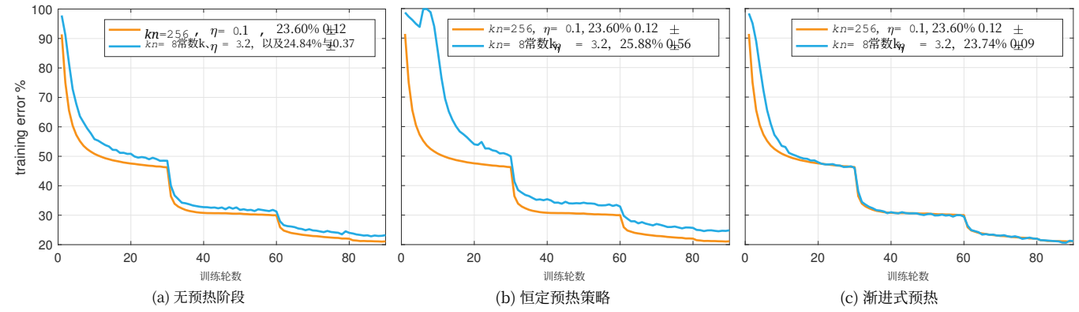


Figure 3. **Training error *vs.* minibatch size.** Training error curves for the 256 minibatch baseline and larger minibatches using gradual warmup and the linear scaling rule. Note how the training curves closely match the baseline (aside from the warmup period) up through 8k minibatches. *Validation* error (mean±std of 5 runs) is shown in the legend, along with minibatch size kn and reference learning rate η.



(a) 无预热阶段 (b) 恒定预热策略 (c) 渐进式预热

图2。**预热阶段**。该图展示了在采用不同预热策略且小批量大小为8192时的训练误差曲线，这些曲线是与小批量大小为256时的情况进行对比的。图例中还显示了验证误差（5次实验的平均值及标准差），以及小批量大小 kn 和参考学习率 η。

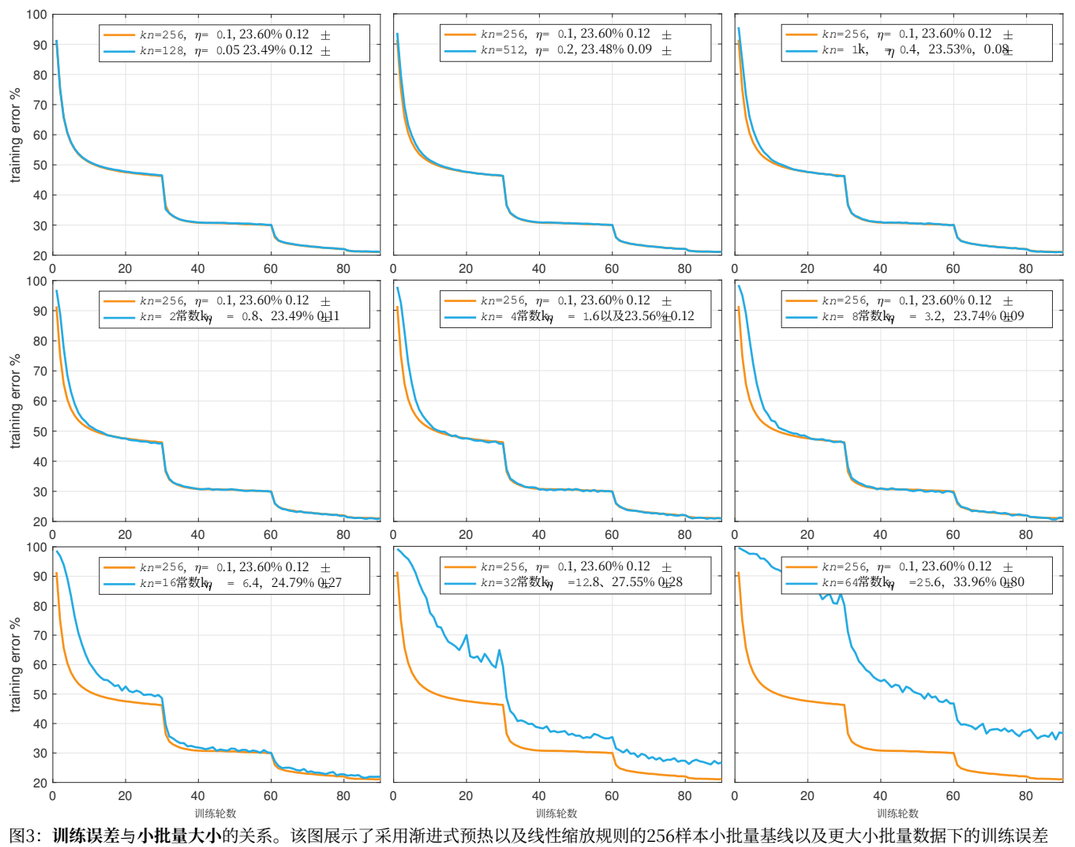


图3：**训练误差与小批量大小**的关系。该图展示了采用渐进式预热以及线性缩放规则的256样本小批量基线以及更大小批量数据下的训练误差曲线。可以看出，在预热阶段之外，直到8k批量大小时，这些训练曲线都与基线曲线高度吻合。验证误差（5次实验的平均值±标准差）会在图例中显示，同时还会标明小批量大小 kn 以及参考学习率 η。

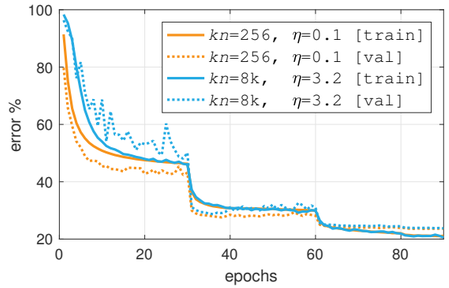


Figure 4. **Training and validation curves** for large minibatch SGD with gradual warmup *vs*. small minibatch SGD. Both sets of curves match closely after training for sufficient epochs. We note that the BN statistics (for inference only) are computed using *running* average, which is updated less frequently with a large minibatch and thus is noisier in early training (this explains the larger variation of the validation error in early epochs).

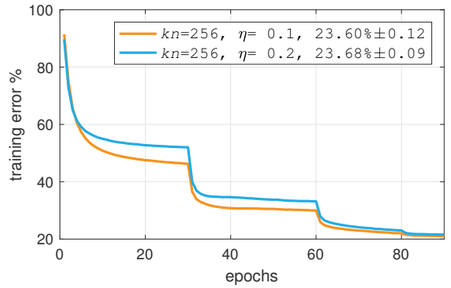


Figure 5. **Training curves for small minibatches with different learning rates η.** As expected, changing η results in curves that *do not match*. This is in contrast to changing batch-size (and linearly scaling η), which results in curves that *do match*, *e.g*. see Figure 3.

5.3. Analysis Experiments

Minibatch size *vs*. error. Figure 1 (page 1) shows top-1 validation error for models trained with minibatch sizes ranging from of 64 to 65536 (64k). For all models we used the linear scaling rule and set the reference learning rate as $\eta = 0.1 \cdot \frac{kn}{256}$. For models with $kn > 256$, we used the gradual warmup strategy always starting with $\eta = 0.1$ and increasing linearly to the reference learning rate after 5 epochs. Figure 1 illustrates that validation error remains stable across a broad range of minibatch sizes, from 64 to 8k, after which it begins to increase. Beyond 64k training diverges when using the linear learning rate scaling rule.[5]

Training curves for various minibatch sizes. Each of the nine plots in Figure 3 shows the top-1 training error curve for the 256 minibatch baseline (orange) and a second curve corresponding to different size minibatch (blue). Validation errors are shown in the plot legends. As minibatch size increases, all training curves show some divergence from the baseline at the start of training. However, in the cases where the final validation error closely matches the baseline ($kn \leq$ 8k), the training curves also closely match after the initial epochs. When the validation errors do not match ($kn \geq$ 16k), there is a noticeable gap in the training curves for all epochs. This suggests that when comparing a new setting, the training curves can be used as a reliable proxy for success well before training finishes.

Alternative learning rate rules. Table 2a shows results for multiple learning rates. For small minibatches ($kn = 256$), $\eta = 0.1$ gives best error but slightly smaller or larger η also work well. When applying the linear scaling rule with a minibatch of 8k images, the optimum error is also achieved with $\eta = 0.1 \cdot 32$, showing the successful application of the linear scaling rule. However, in this case results are more sensitive to changing η. In practice we suggest to use a minibatch size that is not close to the breaking point.

Figure 5 shows the training curves of a 256 minibatch using $\eta = 0.1$ or 0.2. It shows that changing the learning rate η in general changes the overall shapes of the training curves, even if the final error is similar. Contrasting this result with the success of the linear scaling rule (that can match both the final error and the training curves when minibatch sizes change) may reveal some underlying invariance maintained between small and large minibatches.

We also show two alternative strategies: keeping η fixed at 0.1 or using $0.1 \cdot \sqrt{32}$ according to the square root scaling rule that was justified theoretically in [21] on grounds that it scales η by the inverse amount of the reduction in the gradient estimator's standard deviation. For fair comparisons we also use gradual warmup for $0.1 \cdot \sqrt{32}$. Both policies work poorly in practice as the results show.

Batch Normalization γ initialization. Table 2b controls for the impact of the new BN γ initialization introduced in §5.1. We show results for minibatch sizes 256 and 8k with the standard BN initialization ($\gamma = 1$ for all BN layers) and with our initialization ($\gamma = 0$ for the final BN layer of each residual block). The results show improved performance with $\gamma = 0$ for both minibatch sizes, and the improvement is slightly larger for the 8k minibatch size. This behavior also suggests that large minibatches are more easily affected by optimization difficulties. We expect that improved optimization and initialization methods will help push the boundary of large minibatch training.

ResNet-101. Results for ResNet-101 [16] are shown in Table 2c. Training ResNet-101 with a batch-size of $kn = 8$k

[5] We note that because of the availability of hardware, we *simulated* distributed training of very large minibatches (≥12k) on a single server by using multiple gradient accumulation steps between SGD updates. We have thoroughly verified that gradient accumulation on a single server yields equivalent results relative to distributed training.

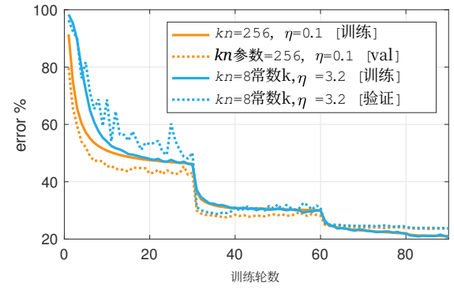


图4展示了采用渐进式预热的大批量SGD算法与小批量SGD算法的**训练曲线和验证曲线**。相比，在经过足够多的训练轮次后，两组的曲线走势十分接近。需要说明的是，此处用于推理的BN统计量是通过逐次计算平均值得出的，而使用大批量时更新频率较低，因此会在训练初期产生更多噪声（这也解释了为何在训练初期验证误差的波动会更大）。

5.3. 分析实验

小批量大小与误差图1 （第1页）显示了使用范围从64到65536（即64K分辨率）不等的不同小批量大小训练所得模型的Top-1验证误差情况。对于所有模型，我们均采用了线性缩放规则，并将参考学习率设定为。而对于那些具有特定结构的模型，我们则采用了渐进式预热策略，始终从某个初始值开始，在5个训练轮次后逐步线性提升至参考学习率。图1表明，当小批量大小在64到8k之间时，验证误差能保持稳定，超过8k后则开始上升；若采用线性学习率缩放规则，训练结果在超过64K分辨率时会出现偏差。

不同小批量大小对应的训练曲线。图中的九个图表3分别展示了256样本小批量基线（橙色曲线）的top-1训练误差变化情况，以及对应不同小批量大小的其他曲线（蓝色曲线）。验证误差则标注在图例中。随着小批量大小的增加，所有训练曲线在训练初期都会出现与基线曲线的一些偏差。不过，当最终验证误差与基线曲线极为接近时（$kn \leq$ 8k），训练曲线在经过最初的几轮训练后也会与基线曲线高度吻合。而若验证误差不相符（$kn \geq$ 16k），则所有训练轮次的曲线之间都会出现明显的差距。这表明在对比新的训练设置时，早在训练结束之前，这些训练曲线就已经能够作为判断是否成功的可靠依据。

其他学习率规则。表格2a展示了不同学习率下的测试结果。对于小批量（$kn = 256$）的情况，

[5] 需要说明的是，由于硬件条件的限制，我们通过在随机梯度下降更新之间采用多次梯度累积的方式，在单台服务器上模拟了使用非常大批量训练数据（≥12k）的分布式训练场景。我们已经充分验证过，单台服务器上的梯度累积方式所获得的训练结果，与分布式训练的结果具有同等准确性。

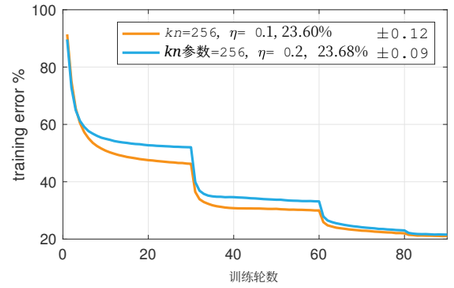


图5.**不同学习率下采用小批量训练的曲线** η。正如预期，改变 η 会导致曲线出现不一致的情况。这与改变批量大小（并应用线性扩展 η）的情形截然不同，后者能使曲线保持一致，例如，详见图3。

$\eta = 0.1$ 能带来最低的误差，而略高或略低的 η 值也同样表现良好。在采用包含8k张图像的小批量数据并应用线性缩放规则时，使用 $\eta = 0.1 \cdot 32$也能得到最优的误差值，这充分体现了该线性缩放规则的有效性。不过在这种情况下，测试结果对 η的变化更为敏感。在实际应用中，我们建议选择远离临界点的小批量大小。

图5 展示了使用 $\eta = 0.1$ 或0的*256*元素小批量训练曲线。*2*从图中可以看出，即便最终误差相近，改变学习率通常也会使训练曲线的整体形态发生变化。将这一结果与线性缩放规则的成功表现——即使小批量大小发生变化，该规则仍能同时保持最终误差和训练曲线的一致性——对比，或许能够揭示小批量训练与大批量训练之间存在的某些内在不变性。

我们还展示了两种替代策略：将 η 固定为0.1，或者依据在 [21]中从理论上得到验证的平方根缩放规则来设定值，该规则的依据是它按照梯度估计器标准差降低幅度的倒数进行缩放。为保证公平比较，我们也在$0.1 \cdot \sqrt{32}$中采用了渐进式预热技术。但正如实验结果所示，这两种方法在实际应用中的效果都很不佳。

批量归一化 γ 初始化方式。表2b 用于分析在§ γ 5.1节中引入的新的批量归一化初始化方式对性能的影响。我们展示了使用标准批量归一化初始化方式（$\gamma = 1$ 适用于所有批量归一化层）以及我们自行设计的初始化方式（$\gamma = 0$ 仅用于每个残差块的最后一个批量归一化层）时，小批量大小为256和8k时的实验结果。实验结果表明，两种小批量大小下的性能均有提升，且8k批量大小下的提升幅度略更大。这一现象也表明，大批量训练更容易受到优化难题的影响。我们预计，更优的优化算法与初始化方法将有助于进一步拓展大批量训练的应用边界。

ResNet-101网络。ResNet-101的实验结果如101 [16] 表2c所示。该模型在批量大小为 $kn = 8$k的情况下进行训练

kn	η	top-1 error (%)
256	0.05	23.92 ±0.10
256	0.10	23.60 ±0.12
256	0.20	23.68 ±0.09
8k	$0.05 \cdot 32$	24.27 ±0.08
8k	$0.10 \cdot 32$	23.74 ±0.09
8k	$0.20 \cdot 32$	24.05 ±0.18
8k	0.10	41.67 ±0.10
8k	$0.10 \cdot \sqrt{32}$	26.22 ±0.03

(a) **Comparison of learning rate scaling rules.** A reference learning rate of $\eta = 0.1$ works best for $kn = 256$ (23.68% error). The linear scaling rule suggests $\eta = 0.1 \cdot 32$ when $kn = 8$k, which again gives best performance (23.74% error). Other ways of scaling η give worse results.

kn	η	γ-init	top-1 error (%)
256	0.1	1.0	23.84 ±0.18
256	0.1	0.0	23.60 ±0.12
8k	3.2	1.0	24.11 ±0.07
8k	3.2	0.0	23.74 ±0.09

(b) **Batch normalization γ initialization.** Initializing $\gamma = 0$ in the *last* BN layer of each residual block improves results for both small and large minibatches. This initialization leads to better optimization behavior which has a larger positive impact when training with large minibatches.

model type	kn	η	top-1 error (%)
ResNet-101	256	0.1	22.08 ±0.06
ResNet-101	8k	3.2	22.36 ±0.09

(c) **The linear scaling rule applied to ResNet-101.** The difference in error is about 0.3% between small and large minibatch training.

Table 2. **ImageNet classification experiments.** Unless noted all experiments use ResNet-50 and are averaged over 5 trials.

and a linearly scaled $\eta = 3.2$ results in an error of 22.36% *vs*. the $kn = 256$ baseline which achieves 22.08% with $\eta = 0.1$. In other words, ResNet-101 trained with minibatch 8k has a small 0.28% increase in error *vs*. the baseline. It is likely that the minibatch size of 8k lies on the edge of the useful minibatch training regime for ResNet-101, similarly to ResNet-50 (see Figure 1).

The training time of ResNet-101 is 92.5 minutes in our implementation using 256 Tesla P100 GPUs and a minibatch size of 8k. We believe this is a compelling result if the speed-accuracy tradeoff of ResNet-101 is preferred.

ImageNet-5k. Observing the sharp increase in validation error between minibatch sizes of 8k and 16k on ImageNet-1k (Figure 1), a natural question is if the position of this 'elbow' in the error curve is a function of dataset information content. To investigate this question, we adopt the ImageNet-5k dataset suggested by Xie *et al*. [39] that extends ImageNet-1k to 6.8 million images (roughly 5× larger) by adding 4k additional categories from ImageNet-22k [33]. We evaluate the 1k-way classification error on the original ImageNet-1k validation set as in [39].

The minibatch size *vs*. validation error curve for ImageNet-5k is shown in Figure 6. Qualitatively, the curve

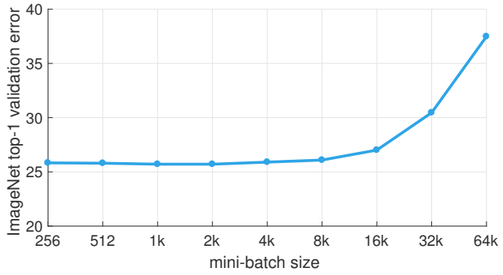


Figure 6. **ImageNet-5k** top-1 validation error *vs*. minibatch size with a fixed 90 epoch training schedule. The curve is qualitatively similar to results on ImageNet-1k (Figure 1) showing that a 5× increase in training data does not lead to a significant change in the maximum effective minibatch size.

ImageNet pre-training			COCO	
kn	η	top-1 error (%)	box AP (%)	mask AP (%)
256	0.1	23.60 ±0.12	35.9 ±0.1	33.9 ±0.1
512	0.2	23.48 ±0.09	35.8 ±0.1	33.8 ±0.2
1k	0.4	23.53 ±0.08	35.9 ±0.2	33.9 ±0.2
2k	0.8	23.49 ±0.11	35.9 ±0.1	33.9 ±0.1
4k	1.6	23.56 ±0.12	35.8 ±0.1	33.8 ±0.1
8k	3.2	23.74 ±0.09	35.8 ±0.1	33.9 ±0.2
16k	6.4	24.79 ±0.27	35.1 ±0.3	33.2 ±0.3

(a) **Transfer learning of large minibatch pre-training to Mask R-CNN.** Box and mask AP (on COCO `minival`) are nearly identical for ResNet-50 models pre-trained with minibatches from 256 to 8k examples. With a minibatch pre-training size of 16k both ImageNet validation error *and* COCO AP deteriorate. This indicates that as long as ImageNet error is matched, large minibatches do not degrade transfer learning performance.

# GPUs	kn	$\eta \cdot 1000$	iterations	box AP (%)	mask AP (%)
1	2	2.5	1,280,000	35.7	33.6
2	4	5.0	640,000	35.7	33.7
4	8	10.0	320,000	35.7	33.5
8	16	20.0	160,000	35.6	33.6

(b) **Linear learning rate scaling applied to Mask R-CNN.** Using the single ResNet-50 model from [16] (thus no std is reported), we train Mask R-CNN using using from 1 to 8 GPUs following the linear learning rate scaling rule. Box and mask AP are nearly identical across all configurations showing the successful generalization of the rule beyond classification.

Table 3. **Object detection on COCO with Mask R-CNN [14].**

is very similar to the ImageNet-1k curve, showing that for practitioners it is unlikely that even a 5× increase in dataset size will automatically lead to a meaningful increase in useable minibatch size. Quantitatively, using an 8k minibatch increases the validation error by 0.26% from 25.83% for a 256 minibatch to 26.09%. An understanding of the precise relationship between generalization error, minibatch size, and dataset information content is open for future work.

5.4. Generalization to Detection and Segmentation

A low error rate on ImageNet is not typically an end goal. Instead, the utility of ImageNet training lies in learn-

kn	η	Top-1误差（%）
256	0.05	23.92 ±0.10
256	0.10	23.60 ±0.12
256	0.20	23.68 ±0.09
8k	$0.05 \cdot 32$	24.27 ±0.08
8k	$0.10 \cdot 32$	23.74 ±0.09
8k	$0.20 \cdot 32$	24.05 ±0.18
8k	0.10	41.67 ±0.10
8k	$0.10 \cdot \sqrt{32}$	26.22 ±0.03

(a) **学习率缩放规则的对比**。当使用 $\eta = 0.1$ 作为参考学习率时，其在 $kn = 256$ 中的表现最佳，对应的误差为23.68%。线性缩放规则建议在 $kn = 8$ k的情况下采用 $\eta = 0.1 \cdot 32$，此时也能获得最优性能，误差同样为23.74%。而其他缩放方式 η 则会导致更差的实验结果。

kn	η	γ-init	Top-1误差（%）
256	0.1	1.0	23.84 ±0.18
256	0.1	0.0	23.60 ±0.12
8k	3.2	1.0	24.11 ±0.07
8k	3.2	0.0	23.74 ±0.09

(b) **批量归一化 γ 初始化。在每个残差块的最后一个**批量归一化层中进行初始化，无论是对小批量还是大批量训练而言，都能提升训练效果。这种初始化方式有助于实现更优的优化过程，而在使用大批量训练时其积极影响更为显著。

模型类型	kn	η	Top-1误差（%）
ResNet-101网络	256	0.1	22.08 ±0.06
ResNet-101网络	8k	3.2	22.36 ±0.09

(c) **应用于ResNet-101网络的线性缩放规则**。 在采用小批量与大规模小批量进行训练时，其误差差异约为0.3%。

表2。**ImageNet分类任务实验结果**。除非另有说明，所有实验均采用ResNet-50模型，并对5次重复试验的结果取平均值。

经过线性缩放后的 $\eta = 3.2$ 测试结果显示误差为22.36%，而基准曲线 $kn = 256$ 的误差则为22.08%。换言之，使用8k批量大小对ResNet-101进行小批量训练时，其误差仅比基准曲线高出0.28%。这表明*8k*作为小批量大小，很可能处于适合*ResNet-101*训练的边界位置，这一情况与*ResNet-50*类似（见图*1*）。

在采用256块Tesla P100显卡且小批量大小为8k的配置下，我们的实现中ResNet-101的训练时间约为92.5分钟。如果我们更看重ResNet-101在速度与精度之间的平衡表现，这一结果无疑颇具说服力。

ImageNet-5k数据集。从ImageNet-1k数据集上可以看出，当小批量大小从8k增至16K分辨率时，验证误差会出现显著上升（见图1）。由此自然会引发这样一个问题：误差曲线中出现这种‘拐点’的位置是否与数据集的内容有关？为探究这一问题，我们采用了Xie等人推荐的ImageNet-5k数据集等。该数据集通过从ImageNet-22k中新增4K分辨率的类别，将ImageNet-1k的数据量扩展到了680万张图像，规模大约 5×更大 [33]。我们按照 [39]中的方法，评估了在原始ImageNet-1k验证集上进行的1k分类任务的误差情况。

不同小批量大小对应的验证误差曲线为Figure 6中展示了ImageNet-5k的数据情况。从整体趋势来看，该曲线

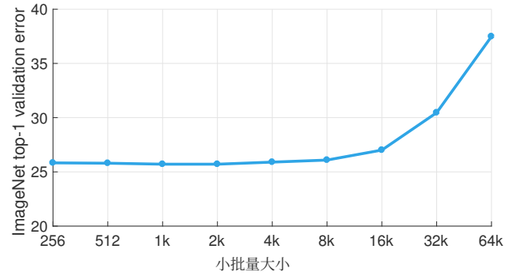


图6。**ImageNet-5k** Top-1验证误差与。在采用固定训练轮次且训练计划不变的条件下，不同小批量大小对应的验证误差曲线在形态上与在ImageNet-1k数据集上得到的结果（见图1）相似，这说明增加训练数据量并不会导致最大有效小批量大小出现显著变化。

ImageNet预训练			COCO	
kn	η	Top-1误差（%）	框检测精度值（%）	掩码检测精度值（%）
256	0.1	23.60 ±0.12	35.9 ±0.1	33.9 ±0.1
512	0.2	23.48 ±0.09	35.8 ±0.1	33.8 ±0.2
1k	0.4	23.53 ±0.08	35.9 ±0.2	33.9 ±0.2
2k	0.8	23.49 ±0.11	35.9 ±0.1	33.9 ±0.1
4k	1.6	23.56 ±0.12	35.8 ±0.1	33.8 ±0.1
8k	3.2	23.74 ±0.09	35.8 ±0.1	33.9 ±0.2
16k	6.4	24.79 ±0.27	35.1 ±0.3	33.2 ±0.3

(a) **将大批量训练的预训练结果迁移应用到Mask R-CNN模型中**。对于使用256到8k个样本作为小批量数据进行预训练的ResNet-50模型而言，其框检测与掩码检测的平均精度值在COCO数据集上几乎完全相同。而当预训练所使用的小批量大小达到16K分辨率时，ImageNet数据集的验证误差以及COCO数据集的平均精度值都会出现下降。这一现象表明，只要ImageNet数据集上的误差保持在相同水平，大批量训练并不会降低迁移学习的性能。

# 图形处理器	kn	$\eta \cdot 1000$ 次迭代		框检测精度值（%）	掩码检测精度值（%）
1	2	2.5	1,280,000	35.7	33.6
2	4	5.0	640,000	35.7	33.7
4	8	10.0	320,000	35.7	33.5
8	16	20.0	160,000	35.6	33.6

(b) **此处采用了针对Mask R-CNN的线性学习率缩放策略**。我们使用来自 [16] <style id='8'>的单一ResNet-50模型（因此未报告标准差），并依据线性学习率缩放规则，在1到8台图形处理器上对Mask R-CNN进行训练。在所有配置下，框检测精度值与掩码检测精度值几乎完全相同，这体现了该策略在分类任务之外的良好泛化能力。

表3。**使用Mask R-CNN [14]在COCO数据集上进行的物体检测结果**。

其曲线与ImageNet-1k数据集的曲线极为相似，这表明对于实际应用而言，即便数据集规模增加 5×，也不太可能自动带来可用小批量大小的显著提升。从数值上看，采用8k批量大小时，验证误差会从使用256批量大小时的25.83%上升0.26%，达到26.09%。未来仍有必要进一步研究泛化误差、小批量大小以及数据集信息量之间的精确关联。

5.4. 向物体检测与分割任务的泛化

在ImageNet数据集上获得较低的错误率通常并非最终目标。实际上，使用该数据集进行训练的价值在于学习那些能够良好迁移或泛化到相关任务中的优质特征。

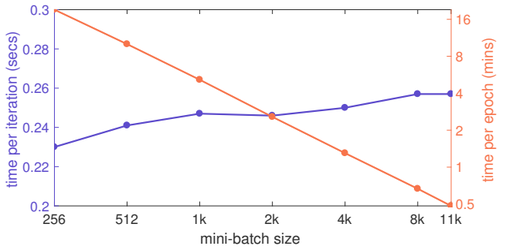


Figure 7. **Distributed synchronous SGD timing.** Time per iteration (seconds) and time per ImageNet epoch (minutes) for training with different minibatch sizes. The baseline ($kn = 256$) uses 8 GPUs in a single server , while all other training runs distribute training over ($kn/256$) server. With 352 GPUs (44 servers) our implementation completes one pass over all ~1.28 million ImageNet training images in about 30 seconds.

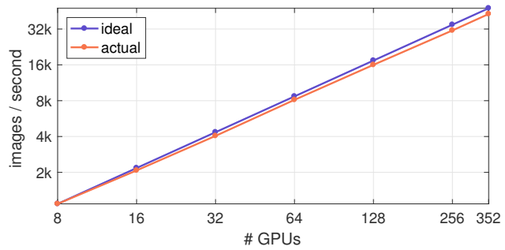


Figure 8. **Distributed synchronous SGD throughput.** The small overhead when moving from a single server with 8 GPUs to multi-server distributed training (Figure 7, blue curve) results in linear throughput scaling that is marginally below ideal scaling (~90% efficiency). Most of the allreduce communication time is hidden by pipelining allreduce operations with gradient computation. Moreover, this is achieved with commodity Ethernet hardware.

ing good features that transfer, or generalize well, to related tasks. A question of key importance is if the features learned with large minibatches generalize as well as the features learned with small minibatches?

To test this, we adopt the object detection and instance segmentation tasks on COCO [27] as these advanced perception tasks benefit substantially from ImageNet pre-training [10]. We use the recently developed Mask R-CNN [14] system that is capable of learning to detect and segment object instances. We follow all of the hyper-parameter settings used in [14] and only change the ResNet-50 model used to initialize Mask R-CNN training. We train Mask R-CNN on the COCO `trainval35k` split and report results on the 5k image `minival` split used in [14].

It is interesting to note that the concept of minibatch size in Mask R-CNN is different from the classification setting. As an extension of the *image-centric* Fast/Faster R-CNN [9, 31], Mask R-CNN exhibits *different minibatch sizes for different layers*: the network backbone uses two images (per GPU), but each image contributes 512 Regions-of-Interest for computing classification (multinomial cross-entropy), bounding-box regression (smooth-L1/Huber), and pixel-wise mask (28×28 binomial cross-entropy) losses. This diverse set of minibatch sizes and loss functions provides a good test case to the robustness of our approach.

Transfer learning from large minibatch pre-training. To test how large minibatch *pre-training* effects Mask R-CNN, we take ResNet-50 models trained on ImageNet-1k with 256 to 16k minibatches and use them to initialize Mask R-CNN training. For each minibatch size we pre-train 5 models and then train Mask R-CNN using all 5 models on COCO (35 models total). We report the mean box and mask APs, averaged over the 5 trials, in Table 3a. The results show that as long as ImageNet validation error is kept low, which is true up to 8k batch size, generalization to object detection matches the AP of the small minibatch baseline. We emphasize that we observed *no generalization issues* when transferring across datasets (from ImageNet to COCO) and across tasks (from classification to detection/segmentation) using models trained with large minibatches.

Linear scaling rule applied to Mask R-CNN. We also show evidence of the generality of the linear scaling rule using Mask R-CNN. In fact, this rule was already used without explicit discussion in [16] and was applied effectively as the default Mask R-CNN training scheme when using 8 GPUs. Table 3b provides experimental results showing that when training with 1, 2, 4, or 8 GPUs the linear learning rate rule results in constant box and mask AP. For these experiments, we initialize Mask R-CNN from the released MSRA ResNet-50 model, as was done in [14].

5.5. Run Time

Figure 7 shows two visualizations of the run time characteristics of our system. The blue curve is the time per iteration as minibatch size varies from 256 to 11264 (11k). Notably this curve is relatively flat and the time per iteration increases only 12% while scaling the minibatch size by 44×. Visualized another way, the orange curve shows the approximately linear decrease in time per epoch from over 16 minutes to just 30 seconds. Run time performance can also be viewed in terms of throughput (images / second), as shown in Figure 8. Relative to a perfectly efficient extrapolation of the 8 GPU baseline, our implementation achieves ~90% scaling efficiency.

Acknowledgements. We would like to thank Leon Bottou for helpful discussions on theoretical background, Jerry Pan and Christian Puhrsch for discussions on efficient data loading, Andrew Dye for help with debugging distributed training, and Kevin Lee, Brian Dodds, Jia Ning, Koh Yew Thoon, Micah Harris, and John Volk for Big Basin and hardware support.

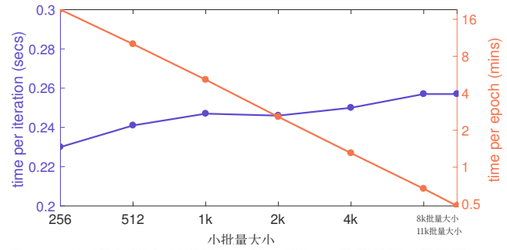


图7。**分布式同步随机梯度下降的耗时情况**。该图展示了使用不同小批量大小进行训练时，每次迭代的耗时（秒）以及每个ImageNet训练轮次的耗时（分钟）。基准曲线（$kn = 256$）是在单台服务器上启用8块GPU实现的，而其余所有训练任务则是在($kn/256$)台服务器上分布式地完成训练。借助352块GPU（分布在44台服务器上），我们的实现仅需大约30秒即可完成对所有 ~1.28百万张ImageNet训练图像的遍历处理。

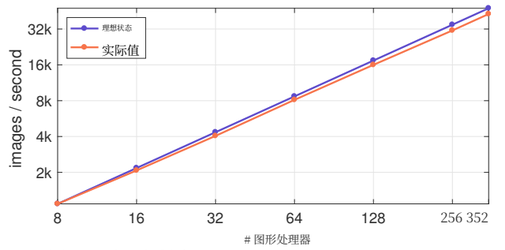


图8 **分布式同步随机梯度下降的吞吐量**。从配备8块图形处理器的单台服务器转向多服务器分布式训练时，所产生的额外开销相对较小（见图7，蓝色曲线），这使得吞吐量的增长呈线性趋势，略低于理想的增长水平（%效率）。大部分全量归约操作的通信时间都被通过将全量归约操作与梯度计算进行流水线处理的方式给掩盖了。此外，这一成果是在普通的以太网硬件上实现的。

一个至关重要的问题是：通过大批量训练学到的特征，其泛化能力是否与通过小批量训练学到的特征相当？

为验证这一方法的有效性，我们选择了COCO数据集上的物体检测与实例分割任务作为测试对象 [27]，因为这类高级感知任务能够从ImageNet预训练中显著受益 [10]。我们采用了最新开发的Mask R-CNN[14] 模型，该模型具备学习识别并分割物体实例的能力。我们在设置上完全沿用了 [14] 中的超参数配置，仅将用于初始化Mask R-CNN训练的模型从其他型号替换为了ResNet-50模型。我们在COCO数据集的`trainval35k` 划分版本上对Mask R-CNN进行训练，并在 [14]中使用的5k图像`minival` 划分版本上输出测试结果。

值得关注的是，Mask R-CNN中所采用的小批量大小概念与分类任务中的设定有所不同。作为以图像为中心的Fast/Faster R-CNN的扩展版本， [9, 31], Mask R-CNN会为不同层设置不同的小批量大小：网络主干结构每台图形处理器会使用两张图像，而每张图像会分别生成512个感兴趣区域，用于计算分类任务（多项式交叉熵）、边界框回归任务（平滑L1/Huber损失）以及像素级掩码任务（28×28 二项式交叉熵损失）。这种多样化的批量大小组合与损失函数搭配，为我们方法的实际稳健性提供了良好的测试依据。

基于大批量训练的迁移学习。为测试大批量 预训练 对Mask R-CNN的影响，我们选用在ImageNet-1k数据集上使用256到16K分辨率的小批量数据训练得到的ResNet-50模型，以此作为Mask R-CNN训练的初始参数。对于每一种不同的小批量大小，我们会预训练5个模型，随后利用这5个模型在COCO数据集上继续训练Mask R-CNN（总计35个模型）。我们在表 3a中给出了5次实验的平均框检测精度和掩码检测精度值。实验结果表明，只要保持ImageNet验证误差处于较低水平——在8k批量大小之前这一条件始终成立——模型在物体检测任务上的泛化能力就能与小样本小批量基线模型的平均精度相媲美。需要特别说明的是，使用经过大批量训练的模型在不同数据集（从ImageNet到COCO）以及不同任务类型（从分类任务到检测/分割任务）之间进行迁移时，我们并未观察到 **任何泛化问题**

检测任务的平均精度与小样本小批量基线曲线相当。我们需要强调的是，使用大批量训练得到的模型在跨数据集迁移（从ImageNet到COCO）以及跨任务迁移（从分类任务到检测/分割任务）时，并未出现泛化问题。

针对Mask R-CNN应用了线性缩放规则。我们还通过Mask R-CNN展示了该线性缩放规则普遍适用性的证据。事实上，这一规则早在 [16]中就被使用过，且在使用8台图形处理器时还被作为默认的Mask R-CNN训练方案有效应用。如3b 表所示，实验结果表明，无论使用1台、2台、4台还是8台图形处理器，遵循线性学习率规则都能保持框检测和掩码检测的精度值不变。在这些实验中，我们与 [14]中的做法一致，从已公开的MSRA ResNet-50模型出发对Mask R-CNN进行了初始化。

5.5. 运行时间

图7 展示了我们系统运行时间特性的两种可视化图表。蓝色曲线表示在不同小批量大小（从256到11264，即11千）下的每次迭代所需时间。值得注意的是，这条曲线相对较为平坦，即便小批量大小发生变化，每次迭代的时间也仅会增加12%。另一种呈现方式是，橙色曲线显示了每轮训练所需时间的近似线性下降趋势，时间从16分钟以上降至仅30秒左右。如图8所示，运行时间性能还可以通过吞吐量（每秒处理的图像数量）来衡量。相较于8台图形处理器基准曲线下的理想高效外推结果，我们的实现方案能够达到~90%的缩放效率。

致谢。我们想感谢Leon Bottou在理论基础方面给予的宝贵指导，感谢Jerry Pan和Christian Puhrsch在高效数据加载方面的建议，感谢An-drew Dye在分布式训练调试方面提供的帮助，同时也要感谢Kevin Lee、Brian Dodds、Jia Ning、Koh Yew Thoon、Micah Harris以及John Volk为Big Basin硬件支持所做出的贡献。

References

[1] J. Bagga, H. Morsy, and Z. Yao. Opening designs for 6-pack and Wedge 100. https://code.facebook.com/posts/203733993317833/opening-designs-for-6-pack-and-wedge-100, 2016.

[2] M. Barnett, L. Shuler, R. van De Geijn, S. Gupta, D. G. Payne, and J. Watts. Interprocessor collective communication library (intercom). In *Scalable High-Performance Computing Conference*, 1994.

[3] L. Bottou. Curiously fast convergence of some stochastic gradient descent algorithms. Unpublished open problem offered to the attendance of the SLDS 2009 conference, 2009.

[4] L. Bottou, F. E. Curtis, and J. Nocedal. Opt. methods for large-scale machine learning. *arXiv:1606.04838*, 2016.

[5] J. Chen, X. Pan, R. Monga, S. Bengio, and R. Jozefowicz. Revisiting Distributed Synchronous SGD. *arXiv:1604.00981*, 2016.

[6] K. Chen and Q. Huo. Scalable training of deep learning machines by incremental block training with intra-block parallel optimization and blockwise model-update filtering. In *ICASSP*, 2016.

[7] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. Kuksa. Natural language processing (almost) from scratch. *JMLR*, 2011.

[8] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, and T. Darrell. Decaf: A deep convolutional activation feature for generic visual recognition. In *ICML*, 2014.

[9] R. Girshick. Fast R-CNN. In *ICCV*, 2015.

[10] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014.

[11] W. Gropp, E. Lusk, and A. Skjellum. *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. MIT Press, Cambridge, MA, 1999.

[12] S. Gross and M. Wilber. Training and investigating Residual Nets. https://github.com/facebook/fb.resnet.torch, 2016.

[13] M. Gürbüzbalaban, A. Ozdaglar, and P. Parrilo. Why random reshuffling beats stochastic gradient descent. *arXiv:1510.08560*, 2015.

[14] K. He, G. Gkioxari, P. Dollár, and R. Girshick. Mask R-CNN. *arXiv:1703.06870*, 2017.

[15] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *ICCV*, 2015.

[16] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[17] G. Hinton, L. Deng, D. Yu, G. E. Dahl, A.-r. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. N. Sainath, et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine*, 2012.

[18] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *arXiv:1510.08560*, 2016.

[19] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[20] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *ICLR*, 2017.

[21] A. Krizhevsky. One weird trick for parallelizing convolutional neural networks. *arXiv:1404.5997*, 2014.

[22] A. Krizhevsky, I. Sutskever, and G. Hinton. ImageNet classification with deep convolutional neural nets. In *NIPS*, 2012.

[23] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1989.

[24] K. Lee. Introducing Big Basin: Our next-generation AI hardware. https://code.facebook.com/posts/1835166200089399/introducing-big-basin, 2017.

[25] M. Li. *Scaling Distributed Machine Learning with System and Algorithm Co-design*. PhD thesis, Carnegie Mellon University, 2017.

[26] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *CVPR*, 2017.

[27] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *ECCV*. 2014.

[28] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015.

[29] Y. Nesterov. *Introductory lectures on convex optimization: A basic course*. Springer, 2004.

[30] R. Rabenseifner. Optimization of collective reduction operations. In *ICCS*. Springer, 2004.

[31] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NIPS*, 2015.

[32] H. Robbins and S. Monro. A stochastic approximation method. *The annals of mathematical statistics*, 1951.

[33] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *IJCV*, 2015.

[34] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. In *ICLR*, 2014.

[35] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[36] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *CVPR*, 2015.

[37] R. Thakur, R. Rabenseifner, and W. Gropp. Optimization of collective comm. operations in MPICH. *IJHPCA*, 2005.

[38] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv:1609.08144*, 2016.

[39] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He. Aggregated residual transformations for deep neural networks. In *CVPR*, 2017.

[40] W. Xiong, J. Droppo, X. Huang, F. Seide, M. Seltzer, A. Stolcke, D. Yu, and G. Zweig. The Microsoft 2016 Conversational Speech Recognition System. *arXiv:1609.03528*, 2016.

[41] M. D. Zeiler and R. Fergus. Visualizing and understanding convolutional neural networks. In *ECCV*, 2014.

参考文献

[1]J. Bagga、H. Morsy和Z. Yao所著的《6-pack和Wedge 100的开启设计》。来源: https://code.facebook.com/posts/203733993317833/opening-designs-for-6-pack-and-wedge-100, 2016年。[2] M. Barnett、L. Shuler、R. van De Geijn、S. Gupta、D. G. Payne和J. Watts撰写的《处理器间集合通信库（intercom）》，发表于1994年的可扩展高性能计算会议论文集。[3] L. Bottou提出的《某些随机梯度下降算法为何能快速收敛》，该未发表的研究问题曾在2009年的SLDS会议上提出，2009年。[4] L. Bottou、F. E. Curtis和J. Nocedal合作的《大规模机器学习的优化方法》，发布于arXiv预印本平台，编号为1606.04838，2016年。[5] J. Chen、X. Pan、R. Monga、S. Bengio和R. Joze-fowicz撰写的《重新审视分布式同步随机梯度下降》，发布于arXiv预印本平台，编号为1604.00981，2016年。[6] K. Chen和Q. Huo研究的《通过增量块训练结合块内并行优化与分块模型更新过滤来实现深度学习模型的可扩展训练》，发表于2016年的ICASSP会议论文集。[7] R. Collobert、J. Weston、L. Bottou、M. Karlen、K. Kavukcuoglu和P. Kuksa合作的《几乎从零开始的自然语言处理》，发表于2011年的JMLR期刊。[8] J. Donahue、Y. Jia、O. Vinyals、J. Hoffman、N. Zhang、E. Tzeng和T. Darrell开发的Decaf模型，这是一种用于通用视觉识别的深度卷积激活特征，发表于2014年的ICML会议论文集。[9] R. Girshick提出的Fast R-CNN模型，发表于2015年的ICCV会议论文集。[10] R. Girshick、J. Donahue、T. Darrell和J. Malik构建的丰富特征层次结构，可用于实现精准的物体检测与语义分割，发表于2014年的CVPR会议论文集。[11] W. Gropp、E. Lusk和A. Skjellum所著的《使用MPI：基于消息传递接口的便携式并行编程》，由MIT Press于1999年在马萨诸塞州剑桥市出版。[12] S. Gross和M. Wilber编写的《残差网络的训练与研究》，相关代码可见https://github.com/facebook/fb.resnet.torch，2016年。[13] M. G¨urb¨uzbalaban、A. Ozdaglar和P. Parrilo研究的《为何随机重排比随机梯度下降更有效》，发布于arXiv预印本平台，编号为1510.08560，2015年。[14] K. He、G. Gkioxari、P. Doll´ar和R. Girshick提出的Mask R-CNN模型，发布于arXiv预印本平台，编号为1703.06870，2017年。´[15] K. He、X. Zhang、S. Ren和J. Sun深入研究的整流器技术，该技术帮助他们在ImageNet分类任务上取得了超越人类水平的性能，发表于2015年的ICCV会议论文集。[16] K. He、X. Zhang、S. Ren和J. Sun提出的深度残差学习方法，用于图像识别任务，发表于2016年的CVPR会议论文集。[17] G. Hinton、L. Deng、D. Yu、G. E. Dahl、A.-r. Mohamed、N. Jaitly、A. Senior、V. Vanhoucke、P. Nguyen、T. N. Sainath等人合作的论文《用于语音识别中声学建模的深度神经网络：四个研究团队的共同见解》，发表于2012年的IEEE信号处理杂志。[18] I. Hubara、M. Courbariaux、D. Soudry、R. El-Yaniv和Y. Bengio研究的量化神经网络，即利用低精度权重和激活值来训练神经网络的相关技术，发布于arXiv预印本平台，编号为1510.08560，2016年。[19] S. Ioffe和C. Szegedy提出的批量归一化技术，通过减少内部协变量偏移来加速深度神经网络的训练，发表于2015年的ICML会议论文集。

[20] N. S. Keskar、D. Mudigere、J. Nocedal、M. Smelyanskiy和P. T. P. Tang所著的《深度学习中的大批量训练：泛化差距与尖锐极小值》发表于2017年的ICLR会议。[21] A. Krizhevsky撰写的《并行化卷积神经网络的一个巧妙技巧》发表在2014年的arXiv预印本平台，编号为1404.5997。[22] A. Krizhevsky、I. Sutskever和G. Hinton共同完成的《利用深度卷积神经网络进行ImageNet分类》发表于2012年的NIPS会议。[23] Y. LeCun等人将反向传播算法应用于手写邮政编码识别任务，相关研究成果发表于1989年的《神经计算》期刊。[24] K. Lee在2017年通过链接 https://code.facebook.com/posts/1835166200089399/introducing-big-basin介绍了他们推出的新一代AI硬件Big Basin。[25] M. Li在其2017年的卡内基梅隆大学博士论文中探讨了如何通过系统与算法协同设计来实现分布式机器学习的规模化发展。[26] T.-Y. Lin、P. Doll´ar、R. Girshick、K. He、B. Hariharan和S. Belongie提出的特征金字塔网络被用于物体检测，相关论文发表于2017年的CVPR会议。´[27] T.-Y. Lin、M. Maire、S. Belongie、J. Hays、P. Perona、D. Ramanan、P. Doll´ar和C. L. Zitnick开发的Microsoft COCO数据集聚焦于上下文中的常见物体，相关成果发表于2014年的ECCV会议。´[28] J. Long、E. Shelhamer和T. Darrell提出的全卷积网络被用于语义分割任务，相关论文发表于2015年的CVPR会议。[29] Y. Nesterov所著的《凸优化入门：基础课程》由Springer出版社在2004年出版。[30] R. Rabenseifner在2004年的Springer出版的《ICCS会议论文集》中研究了集体归约操作的优化问题。[31] S. Ren、K. He、R. Girshick和J. Sun开发的Faster R-CNN模型借助区域提议网络实现了更高效的实时物体检测，相关论文发表于2015年的NIPS会议。[32] H. Robbins和S. Monro在1951年的《数学统计年刊》上发表了关于随机逼近方法的论述。[33] O. Russakovy、J. Deng、H. Su等人在2015年的IJCV会议上提出了ImageNet大规模视觉识别挑战赛。[34] P. Sermanet、D. Eigen、X. Zhang、M. Mathieu、R. Fergus和Y. LeCun在2014年的ICLR会议上展示了Overfeat算法，该算法结合了卷积网络实现识别、定位与检测的一体化功能。[35] K. Simonyan和A. Zisserman在2015年的ICLR会议上提出了用于大规模图像识别的超深卷积网络。[36] C. Szegedy、W. Liu、Y. Jia等人于2015年在CVPR会议上发表了关于通过不断加深卷积层来提升模型性能的研究。[37] R. Thakur、R. Rabenseifner和W. Gropp在2005年的IJHPCA会议上研究了在MPICH并行计算框架中集体通信操作的优化问题。[38] Y. Wu等人开发的谷歌神经机器翻译系统在2016年的arXiv预印本平台上发布，编号为1609.08144，该系统旨在缩小人类翻译与机器翻译之间的差距。[39] S. Xie、R. Girshick、P. Doll´ar、Z. Tu和K. He在2017年的CVPR会议上提出了用于深度神经网络的聚合残差变换方法。´[40] W. Xiong、J. Droppo、X. Huang、F. Seide、M. Seltzer、A. Stolcke、D. Yu和G. Zweig在2016年的arXiv预印本平台上发布了Microsoft 2016对话式语音识别系统，编号为1609.03528。[41] M. D. Zeiler和R. Fergus在2014年的ECCV会议上发表了关于可视化与理解卷积神经网络的相关研究。